



UNIVERSIDAD DE BURGOS  
ESCUELA POLITÉCNICA SUPERIOR  
Grado en Ingeniería Informática



TFG del Grado en Ingeniería  
Informática

**VRLearnTobot Laboratorio  
Virtual de programación y  
robótica**



Presentado por Luis Ignacio de Luna Gómez  
en Universidad de Burgos — 27 de junio  
de 2025

Tutor: Carlos López Nozal







UNIVERSIDAD DE BURGOS  
ESCUELA POLITÉCNICA SUPERIOR  
Grado en Ingeniería Informática



D. Carlos López Nozal, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos..

Expone:

Que el alumno D. Luis Ignacio de Luna Gómez, con DNI 34809179J, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado VRLearn-Tobot Laboratorio Virtual de programación y robótica

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 27 de junio de 2025

Vº. Bº. del Tutor:

D. nombre tutor D. Carlos López Nozal





## Resumen

El presente Trabajo Fin de Grado aborda el diseño e implementación de un prototipo de laboratorio virtual 3D para la enseñanza de los fundamentos de la programación y la robótica, con un enfoque en la accesibilidad y la motivación de estudiantes en etapas preuniversitarias. Desarrollado en el motor Unity con C#, el proyecto introduce un sistema de programación híbrido que combina una representación visual de la estructura del programa con la escritura de comandos textuales.

La principal contribución técnica es la creación de un Lenguaje de Dominio Específico (DSL) simple, junto con un parser que traduce el código fuente a una secuencia de instrucciones ejecutables, y un motor de simulación que interpreta dichas instrucciones para controlar un robot virtual en tiempo real. La arquitectura del sistema, inspirada en el patrón Modelo-Vista-Controlador (MVC), gestiona la interacción del usuario en un entorno de desafíos gamificados. El proyecto culmina en un Producto Mínimo Viable (MVP) que no solo demuestra la viabilidad técnica de la solución, sino que también funciona como un puente pedagógico, facilitando la transición de la programación por bloques a la codificación textual tradicional y alineándose con los Objetivos de Desarrollo Sostenible de democratizar la educación tecnológica.

## Descriptores

Robótica Educativa, Unity, Programación Híbrida, Lenguaje de Dominio Específico (DSL), Simulador 3D, Parser, Intérprete, Pensamiento Computacional, STEAM.

## Abstract

This Final Degree Project addresses the design and implementation of a 3D virtual laboratory prototype for teaching the fundamentals of programming and robotics, with a focus on accessibility and motivation for pre-university students. Developed in the Unity engine with C#, the project introduces a hybrid programming system that combines a visual representation of the program's structure with the writing of textual commands.

The main technical contribution is the creation of a simple Domain-Specific Language (DSL), along with a parser that translates the source code into a sequence of executable instructions, and a simulation engine that interprets these instructions to control a virtual robot in real-time. The system's architecture, inspired by the Model-View-Controller (MVC) pattern, manages user interaction within a gamified challenge-based environment. The project culminates in a Minimum Viable Product (MVP) that not only demonstrates the technical feasibility of the solution but also serves as a pedagogical bridge, facilitating the transition from block-based programming to traditional textual coding, and aligning with the Sustainable Development Goals of democratizing technology education.

## Keywords

Educational Robotics, Unity, Hybrid Programming, Domain-Specific Language (DSL), 3D Simulator, Parser, Interpreter, Computational Thinking, STEAM.



---

# Índice general

---

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| Índice general                                                                | iii |
| Índice de figuras                                                             | v   |
| Índice de tablas                                                              | vi  |
| 1. Introducción                                                               | 1   |
| 2. Objetivos del proyecto                                                     | 5   |
| 2.1. Introducción                                                             | 5   |
| 2.2. Objetivos Funcionales                                                    | 6   |
| 2.3. Objetivos No Funcionales                                                 | 7   |
| 3. Conceptos teóricos                                                         | 9   |
| 3.1. Plataformas de programación visual y kits de robótica                    | 9   |
| 3.2. Arquitectura del sistema de programación                                 | 11  |
| 3.3. El modelo de Bloque (Block)                                              | 13  |
| 3.4. Conexiones (Connection)                                                  | 14  |
| 3.5. Entradas (Input)                                                         | 16  |
| 3.6. Campos (Field)                                                           | 16  |
| 3.7. Variables (VariableMap)                                                  | 17  |
| 3.8. Sistema de ejecución (CSharpRunner e Cmdtors)                            | 17  |
| 3.9. Representación visual y composición de bloques en Unity                  | 18  |
| 3.10. Serialización y persistencia                                            | 20  |
| 3.11. Arquitectura del Prototipo Final: Un Sistema Híbrido                    | 20  |
| 3.12. Objetivos de Desarrollo Sostenible (ODS) y su Aplicación en el Proyecto | 22  |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>4. Técnicas y herramientas</b>                                    | <b>25</b> |
| 4.1. Metodología de desarrollo: Agile con SCRUM . . . . .            | 25        |
| 4.2. Herramientas core para el desarrollo de software . . . . .      | 26        |
| 4.3. Persistencia de datos del programa . . . . .                    | 28        |
| 4.4. Herramientas para gestión de proyectos y desarrollo . . . . .   | 29        |
| <b>5. Aspectos relevantes del desarrollo del proyecto</b>            | <b>31</b> |
| 5.1. Metodología de desarrollo: Iteraciones ágiles con SCRUM . .     | 31        |
| 5.2. Diseño e implementación de la arquitectura inicial . . . . .    | 32        |
| 5.3. Detalles clave de implementación . . . . .                      | 34        |
| 5.4. Pivote estratégico y diseño del sistema híbrido final . . . . . | 36        |
| 5.5. Retos técnicos abordados y estrategias de depuración . . . . .  | 39        |
| <b>6. Trabajos relacionados</b>                                      | <b>43</b> |
| 6.1. Plataformas de programación por bloques . . . . .               | 43        |
| 6.2. Simuladores y entornos de robótica educativa . . . . .          | 44        |
| 6.3. Herramientas para el desarrollo de interfaces de bloques . . .  | 45        |
| 6.4. Síntesis de la contribución del proyecto . . . . .              | 46        |
| <b>7. Conclusiones y líneas de trabajo futuras</b>                   | <b>49</b> |
| 7.1. Conclusiones . . . . .                                          | 49        |
| 7.2. Líneas de trabajo futuras . . . . .                             | 50        |
| 7.3. Conclusión Final . . . . .                                      | 54        |
| <b>Bibliografía</b>                                                  | <b>57</b> |

---

## Índice de figuras

---

---

# Índice de tablas

---

|                                                              |    |
|--------------------------------------------------------------|----|
| 3.1. Comparación de Scratch y Blockly . . . . .              | 10 |
| 3.2. Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime . . . . | 10 |

---

# 1. Introducción

---

En la última década, el interés por la enseñanza de la programación y la robótica ha crecido de manera significativa, con especial énfasis en la educación preuniversitaria, dirigida a **niños y adolescentes**. Esto se debe a la creciente necesidad de dotar a las nuevas generaciones de habilidades fundamentales en pensamiento computacional, resolución de problemas y lógica algorítmica, consideradas esenciales para su desarrollo en el mundo digital actual [22]. La alfabetización digital y el pensamiento computacional son habilidades transversales cruciales que promueven la resolución creativa de problemas y preparan a los estudiantes para los desafíos de la sociedad moderna [11, 3].

Entre las metodologías más empleadas en la enseñanza de la programación, la **programación basada en bloques** ha demostrado ser una herramienta pedagógica eficaz. Este paradigma visual facilita la comprensión de la lógica de programación y el flujo de control sin la complejidad de la sintaxis textual, reduciendo la barrera de entrada para estudiantes novatos y promoviendo un aprendizaje más accesible y lúdico [23, 15]. Su efectividad se ha demostrado en diversos contextos educativos, como la incorporación de robótica en el currículo STEM a través de herramientas de bloques como Scratch [7]. Experiencias en la introducción a la programación en la universidad han mostrado que el uso de estos lenguajes ha mejorado significativamente las tasas de aprobación en asignaturas fundamentales de la informática [15].

Actualmente, dos de las plataformas más destacadas en este ámbito son **Scratch y Blockly**. Scratch, desarrollado por el MIT Media Lab, se ha consolidado como una de las herramientas educativas más utilizadas globalmente para la enseñanza de la programación, proporcionando un entorno altamente accesible y motivador [23]. Por otro lado, Blockly, desarrollado por Google,

ofrece una biblioteca de código abierto que permite integrar programación por bloques en diversas plataformas y puede convertir los bloques visuales a lenguajes de programación textuales como JavaScript y Python, ofreciendo así una mayor versatilidad en la generación de código y aplicaciones [15]. Aunque **Scratch utiliza una implementación personalizada basada en los principios de Blockly**, existen diferencias fundamentales entre ambos sistemas, especialmente en términos de flexibilidad y aplicación práctica [2]. La adopción y los avances en este campo se observan con gran desarrollo en diversas regiones de América Latina, donde la robótica educativa se ha posicionado como un medio eficaz para la enseñanza de habilidades STEM en docentes de primaria, evidenciando un compromiso temprano con estas metodologías [20].

El **aprendizaje basado en retos y la inclusión de elementos lúdicos** ha emergido como un enfoque clave en la enseñanza de la programación. Al introducir *desafíos con objetivos claros y un entorno interactivo*, se promueve una motivación intrínseca y un compromiso activo del estudiante en el proceso de aprendizaje, logrando así una experiencia más efectiva y significativa [2]. Estudios recientes confirman que los proyectos de robótica aumentan significativamente la motivación y el engagement de los estudiantes, facilitando un aprendizaje más profundo y aplicado [6]. Además, *los entornos interactivos con la programación (incluso con componentes de IA)* demuestran resultados positivos en el rendimiento académico y la motivación en estudiantes de primaria y secundaria [1]. Es en este cruce entre la rigurosa aplicación de los conocimientos técnicos y la creatividad del diseño, donde reside la oportunidad de desarrollar herramientas capaces de **vincular y emocionar a los estudiantes**, transformando el aprendizaje de la programación en una actividad lúdica y accesible que genere un impacto duradero.

No obstante, a pesar de estos avances metodológicos y didácticos, existen desafíos significativos para la enseñanza de la programación y robótica, especialmente en entornos con acceso limitado a recursos educativos y tecnológicos. La falta de laboratorios físicos y la dependencia de materiales y robots costosos restringen la accesibilidad de estas valiosas herramientas, afectando principalmente a estudiantes en zonas rurales o con restricciones económicas [22, 12]. La inviabilidad de replicar laboratorios con equipamiento sofisticado en contextos de bajo presupuesto y la dificultad de acceder a prácticas con componentes reales impulsan la búsqueda de alternativas flexibles y accesibles [22]. Esta problemática está directamente alineada con los **Objetivos de Desarrollo Sostenible (ODS)** de la Organización de las Naciones Unidas (ONU), en particular el **ODS 4 (Educación de**

**calidad**), que busca garantizar una educación inclusiva, equitativa y de calidad para niños y adolescentes [21]. Se enfoca en que nadie se quede atrás en el acceso a oportunidades educativas, y el **ODS 10 (Reducción de las desigualdades)**, que promueve la reducción de las brechas sociales y económicas entre regiones y poblaciones, buscando asegurar un acceso equitativo al conocimiento y a las oportunidades educativas [21].

Frente a esta problemática, los **entornos de simulación virtual y los laboratorios virtuales de robótica** han demostrado ser una solución excepcionalmente eficaz para *democratizar el acceso* a la enseñanza de la programación y robótica. Estas plataformas permiten a los estudiantes diseñar, construir y programar robots virtuales en un entorno digital, eliminando la necesidad de hardware físico costoso y ofreciendo un espacio de experimentación seguro y flexible. Permiten proporcionar una experiencia similar a la obtenida en un laboratorio de prácticas a través de sistemas de apoyo con gran realismo [16]. El uso de **juegos serios** (serious games) y simulaciones interactivas se ha consolidado como una metodología prometedora para la enseñanza de la programación, al integrar desafíos divertidos y elementos educativos que mejoran la retención del conocimiento e incrementan significativamente la motivación y la participación activa de los estudiantes [19].

El enfoque inicial de este proyecto consideraba la **replicación completa de un sistema de programación puramente visual basado en bloques, similar a Scratch, dentro de Unity**. Se diseñó una arquitectura robusta para la gestión, conexión y ejecución de bloques, sentando unas bases técnicas sólidas. Sin embargo, durante el desarrollo, se constató que la implementación de un motor de layout gráfico completamente dinámico para las vistas de los bloques presentaba una complejidad técnica que desviaba el foco del objetivo principal: la lógica de programación y la simulación del robot. Ante este reto de ingeniería, se tomó una decisión estratégica: pivotar hacia un Producto Mínimo Viable (MVP) que, aun manteniendo la filosofía de simplicidad y la representación visual del programa, se centrara en una interacción textual. Este reenfoque permitió concentrar los esfuerzos en el diseño de un lenguaje específico, su intérprete y la interacción con el entorno 3D.

Si bien el desarrollo de tecnologías en áreas como la simulación 3D o los lenguajes visuales es sólida, la *integración holística y accesible* de estos elementos en un *único sistema orientado a la robótica educativa específica de bajo coste* sigue presentando oportunidades significativas para el desarrollo y la implementación

**de soluciones innovadoras.** La creación de un puente pedagógico entre los sistemas puramente visuales y la programación textual tradicional, mediante un lenguaje de comandos simple y un entorno 3D inmediato, representa una oportunidad significativa. Este proyecto busca aportar valor en este nicho, ofreciendo una herramienta que facilita la transición hacia la codificación escrita junto con la programación por bloques.

En este contexto, el presente **Trabajo Fin de Grado** se orienta al desarrollo de un **prototipo de laboratorio virtual de programación y robótica**. La plataforma está diseñada con el motor de videojuegos **Unity** utilizando **C#**, con el objetivo de ofrecer una alternativa *accesible, interactiva y didáctica* para la metodología **STEAM**. La herramienta introduce un lenguaje de comandos textual y un intérprete propio, diseñados para ser intuitivos y controlar un robot virtual en un entorno 3D. El programa construido por el usuario se representa visualmente como una lista secuencial de instrucciones, manteniendo la claridad de un sistema de bloques pero fomentando la práctica con la escritura de código. El entorno de simulación está inspirado en la funcionalidad de kits como LEGO WeDo 2.0 y está optimizado para su uso en dispositivos Android tipo tablet.

Este proyecto plantea un desafío técnico centrado en el análisis sintáctico (parsing) y la interpretación de un lenguaje a medida. El sistema implementa un pipeline completo: desde la entrada de texto en una interfaz de usuario, su análisis y conversión a una estructura de datos intermedia (la lista de instrucciones), hasta la ejecución de dichas instrucciones en el motor 3D, gestionando el estado del robot y las interacciones con el entorno, como las colisiones.

La estructura de esta memoria detallará, en primer lugar, los objetivos específicos del prototipo. Posteriormente, se presentarán los conceptos teóricos que sustentan el desarrollo, incluyendo la arquitectura del intérprete de comandos y del motor de simulación. A continuación, se describirán las técnicas y herramientas empleadas. El capítulo central detallará los aspectos más relevantes del diseño e implementación del prototipo, seguido de una comparativa con trabajos relacionados. Finalmente, se expondrán las conclusiones obtenidas y las líneas de trabajo futuras para la evolución de este laboratorio virtual.



---

## 2. Objetivos del proyecto

---

### 2.1. Introducción

El presente Trabajo Fin de Grado tiene como objetivo principal el desarrollo de un **prototipo de laboratorio virtual de programación y robótica enfocado en la enseñanza de los fundamentos de la lógica algorítmica a través de un sistema de desafíos interactivos** que permita a los estudiantes aprender programación mediante bloques y código escrito. Este laboratorio ha sido desarrollado con el motor de videojuegos **Unity 6** junto con el lenguaje de programación **CSharp** para la programación de las clases necesarias para que se lleve a cabo su ejecución.

La educación en robótica y programación ha demostrado ser una herramienta clave en el aprendizaje del pensamiento computacional. Sin embargo, uno de los principales desafíos en la enseñanza de la robótica es la dependencia de hardware físico, lo que limita el acceso a estudiantes en contextos de escasos recursos.

Para abordar esta problemática, el laboratorio virtual combinará un **entorno de simulación 3D**, que permitirá visualizar la ejecución de los programas, con un **sistema de programación basado en un lenguaje de comandos textual**. Este sistema se inspira en la robótica educativa, con la intención de simular un robot tipo LEGO WeDo 2.0 virtual, implementándose en este prototipo los mecanismos de movimiento fundamentales.

Para ello, se establecen los siguientes objetivos específicos que se han perseguido y alcanzado, parcial o totalmente, durante el desarrollo del prototipo:

## 2.2. Objetivos Funcionales

Estos objetivos están relacionados con las funcionalidades que el prototipo de laboratorio virtual debe cumplir para ofrecer una experiencia educativa inicial.

### 1. Diseñar y desarrollar un sistema híbrido de programación para robótica educativa.

- Crear un entorno de programación que combine la programación visual por bloques para las estructuras principales del programa, con la programación textual para la definición de secuencias de comandos específicas.
- Implementar el núcleo de un sistema de programación por bloques inspirado en Scratch, enfocado en los elementos esenciales para la estructuración de un programa:
  - **Bloque de evento de inicio:** como punto de partida de la ejecución.
  - **Bloques de programación estructurada:** como bucles o condicionales que sirven de contenedores para otras instrucciones.
  - **Bloques de movimiento:** Bloques específicos que permitan al robot girar o moverse en alguna dirección.
- Desarrollar un Lenguaje de Dominio Específico (DSL) textual para los comandos de bajo nivel del robot (mover, girar), y un **parser** para validar y procesar dichos comandos.

### 2. Integrar el sistema de programación híbrido con un entorno de simulación 3D.

- Implementar un área de trabajo interactiva en Unity donde se puedan arrastrar y conectar los bloques visuales.
- Desarrollar un motor de ejecución que recorra la estructura de bloques y, cuando encuentre un bloque de código textual, invoque al parser y ejecute las instrucciones resultantes en el robot simulado.
- Configurar un entorno de simulación 3D que permita la visualización en tiempo real del comportamiento del robot, respondiendo tanto a la estructura de bloques como a los comandos textuales.

- Implementar un sistema de detección de colisiones que detenga la ejecución y proporcione feedback inmediato al usuario.

### **3. Implementar un prototipo de laboratorio virtual basado en desafíos.**

- Crear un modelo 3D de un robot genérico inspirado en la estética de kits educativos como el robot Lego Wedo 2.0.
- Dotar al robot virtual de las funcionalidades de movimiento y rotación necesarias para completar los desafíos.
- Diseñar un sistema de gestión de desafíos que permita definir objetivos concretos, cargando dinámicamente las instrucciones y la configuración de la escena.
- Asegurar que el prototipo demuestre el flujo completo desde la construcción del programa híbrido hasta la simulación de su resultado en el entorno 3D.

## **2.3. Objetivos No Funcionales**

Estos objetivos se enfocan en las cualidades transversales de la solución, como su accesibilidad, rendimiento y usabilidad.

### **1. Accesibilidad y escalabilidad del prototipo.**

- Desarrollar una plataforma que pueda ejecutarse en dispositivos de hardware común y sin requerimientos técnicos elevados, con el objetivo de priorizar la portabilidad a plataformas Android (Tablets).
- Explorar estrategias de diseño y desarrollo que minimicen la dependencia de equipamiento costoso, haciendo la herramienta accesible a estudiantes en zonas con recursos limitados.

### **2. Investigación y análisis.**

- Investigar y analizar plataformas existentes de programación y simulación robótica (ej. CoSpaces Edu, Robot Virtual Worlds, VEXcode VR y CoderZ) para comprender sus fortalezas y limitaciones.
- Definir qué aspectos de las soluciones existentes pueden ser mejorados o adaptados en la solución propuesta de este TFG, sentando las bases para futuras evoluciones del sistema.

### **3. Adquisición de conocimientos técnicos.**

- Adquirir conocimientos y habilidades en el uso del motor Unity 6 y el lenguaje C# para la creación de entornos interactivos y simulaciones educativas.
- Aplicar buenas prácticas en el diseño de software y en la integración de sistemas de programación visual dentro de un motor de simulación, incluyendo la aplicación de patrones de diseño (MVC, Observador, Fábrica) y la gestión eficiente de estructuras de datos.

---

## 3. Conceptos teóricos

---

La implementación del laboratorio virtual desarrollado en este Trabajo de Fin de Grado se fundamenta en un análisis de diversas plataformas de programación visual y en una arquitectura de software modular. Este capítulo detalla los conceptos teóricos que sustentan el prototipo, comenzando con la arquitectura inicial basada en programación por bloques (que inspiró la estructura del sistema) y culminando en los conceptos clave del sistema híbrido final, que combina bloques y código textual.

### 3.1. Plataformas de programación visual y kits de robótica

#### Comparación entre Scratch y Blockly (Google)

Actualmente, Scratch y Blockly son las plataformas dominantes en el ámbito de la programación visual basada en bloques para la educación. Su análisis comparativo ha sido fundamental para definir la dirección del presente proyecto, como se detalla en la Tabla 3.1:

#### Comparación entre LEGO WeDo 2.0 y LEGO SPIKE Prime

La elección de LEGO WeDo 2.0 como el robot de referencia para la simulación en este proyecto se justifica al analizar sus diferencias fundamentales con otras plataformas de robótica educativa de LEGO, como se detalla en la Tabla 3.2:

| Característica                   | Scratch                                                                  | Blockly                                                                          |
|----------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Paleta de colores</b>         | Fija (según categorías)                                                  | Personalizable                                                                   |
| <b>Categorización de bloques</b> | Movimiento, control, sensores, eventos, operadores, variables, funciones | Modular y personalizable                                                         |
| <b>Exportación de código</b>     | No permite exportar código                                               | Genera código en múltiples lenguajes (JavaScript, Python, Lua)                   |
| <b>Flexibilidad</b>              | Cerrado, no permite modificar bloques                                    | Altamente personalizable                                                         |
| <b>Uso principal</b>             | Aprendizaje de programación visual en educación                          | Desarrollo de interfaces de programación visual para integración con código real |

Tabla 3.1: Comparación de Scratch y Blockly

| Característica                   | LEGO WeDo 2.0                                                | LEGO SPIKE Prime                                                                        |
|----------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Paleta de colores</b>         | Basada en íconos                                             | Adaptación de Scratch, con bloques en formato palabra e íconos                          |
| <b>Categorización de bloques</b> | Basada en hardware (motores, sensores)                       | Similar a Scratch, con bloques específicos para hardware LEGO                           |
| <b>Tipo de bloques</b>           | Bloques verticales con íconos sin palabras                   | Bloques de palabras en horizontal y algunos bloques con íconos                          |
| <b>Exportación de código</b>     | No permite exportar código                                   | No permite exportar código                                                              |
| <b>Flexibilidad</b>              | Cerrado, bloques predefinidos                                | Más flexible que WeDo 2.0, pero limitado al ecosistema LEGO                             |
| <b>Uso principal</b>             | Programación básica para principiantes en robótica educativa | Programación avanzada para proyectos de robótica con mayor capacidad de personalización |

Tabla 3.2: Comparación de LEGO WeDo 2.0 y LEGO SPIKE Prime

LEGO SPIKE Prime es una adaptación de Scratch hecha por LEGO, por lo que mantiene una estructura de bloques similar a Scratch, pero ajustada

a su ecosistema de hardware. LEGO WeDo 2.0 usa una interfaz más sencilla con bloques puramente icónicos (sin palabras), mientras que SPIKE Prime combina texto e íconos.

## 3.2. Arquitectura del sistema de programación

La arquitectura del prototipo se diseñó de manera evolutiva. Inicialmente, se concibió un sistema puramente visual basado en la librería de código abierto UBlockly. A continuación se detalla esta arquitectura de referencia, ya que sus principios (Modelo-Vista, sistema de conexiones, etc.) fueron la base sobre la que se construyó el prototipo, incluso tras el pivote estratégico hacia un modelo híbrido.

### Adaptación de UBlockly al patrón Modelo-Vista-Controlador (MVC)

Para superar las limitaciones de la estructura original de UBlockly, se implementó una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC). Esta separación de responsabilidades, que no estaba explícita en la librería base, fue crucial para organizar el código, facilitar la depuración y sentar las bases para un sistema escalable. La nueva estructura se organizó de la siguiente manera:

- **Modelo:** Representa la lógica de negocio y el estado del programa. Incluye las estructuras de datos que definen los bloques (`Block`), sus conexiones (`Connection1`), entradas (`Input`), campos (`Field`) y la organización del espacio de trabajo (`WorkSpace`), así como la gestión de variables (`VariableMap`) y procedimientos (`ProcedureDB`). Las operaciones de datos se abstraen de la presentación visual.
- **Vista:** Se encarga de la representación visual del Modelo. Incluye todos los elementos de la interfaz de usuario de Unity (`UI.Image`, `TMPPro.TextMeshProUGUI`, `UI.Button`) y la representación 3D del robot (`RobotBehaviour`). Las vistas observan los cambios en el modelo y se actualizan automáticamente sin modificar directamente los datos subyacentes.
- **Controlador:** Esta capa es una de las principales aportaciones del proyecto, creando una serie de controladores que no existían en

UBlockly para manejar la interacción del usuario y la lógica de la aplicación, coordinando las comunicaciones entre Modelos y Vistas. Clases como `WorkspaceController`, `BlockController`, `TollBoxController` operan en esta capa, traduciendo las entradas del usuario y las acciones del programa en manipulaciones del Modelo o actualizaciones de la Vista.

## Espacio de Trabajo (`Workspace`)

El `Workspace` es el contenedor principal que gestiona el estado lógico y la organización de todos los elementos del programa de bloques. Su principal función es mantener la persistencia y disposición de los bloques, variables y procedimientos en el entorno de programación.

### ■ Atributos principales del `Workspace`:

- **Id:** Identificador único para el espacio de trabajo.
- **Options:** Configuraciones que definen el comportamiento global, como el modo de solo lectura o la ejecución sincrónica.
- **TopBlocks:** `List<Block>` de todos los bloques que no están conectados a otros bloques en una pila superior (es decir, bloques en la cima de la jerarquía).
- **BlockDB:** `Dictionary<string, Block>` que almacena todos los bloques presentes en el espacio de trabajo, indexados por su ID único.
- **VariableMap:** Estructura (`VariableMap`) para la gestión de variables de usuario, incluyendo su creación, eliminación y renombrado.
- **ConnectionDBList:** `Dictionary<EConnection, BlockConnectionDB>` que organiza y gestiona las bases de datos de conexiones, optimizando la búsqueda y validación de uniones entre bloques.
- **ProcedureDB:** Base de datos (`ProcedureDB`) que gestiona los procedimientos o funciones definidos por el usuario dentro del espacio de trabajo.

## Ciclo de Vida de Bloques: Creación y Eliminación

En UBlockly, la gestión de la vida de los bloques sigue un ciclo controlado. La `BlockFactory` se encarga de instanciar un `Block` y lo inicializa con su



estructura lógica (**BlockDefinition**), incluyendo sus entradas, campos y conexiones. Este proceso garantiza que cada bloque se configure correctamente desde el momento de su creación.

- **Creación de bloques:** La clase **BlockFactory** permite la instancia-  
ción de un **Block** (sea un bloque real en el **Workspace** o una plan-  
tilla en la **Toolbox**). Durante este proceso, se crean las **Connection**  
(**OutputConnection**, **PreviousConnection**, **NextConnection**) y las  
**Input** (que a su vez contienen **FieldModel** y sus propias **Connection**  
de entrada), asignando siempre la referencia correcta al bloque propie-  
tario (**SourceBlock**).
- **Eliminación de bloques:** La eliminación se coordina a través del  
método **Block.Dispose()**. Este método no solo retira el bloque de  
las bases de datos del **Workspace**, sino que también se encarga de des-  
vincularlo de sus conexiones mediante **Block.UnPlug()** y de eliminar  
recursivamente todos sus bloques hijos conectados (liberando así los  
recursos asociados a la vista y al modelo del bloque). Este proceso  
garantiza una limpieza efectiva de la memoria y la coherencia del  
estado del programa.

### 3.3. El modelo de Bloque (Block)

Cada bloque es una unidad fundamental del programa, representada por una instancia de **Block**. Un **Block** encapsula la información lógica y las interacciones de un bloque.

- **Componentes principales de un Block:**
  - **Connection:** Los puntos de conexión lógicos del bloque.
    - **OutputConnection:** Permite que un bloque que devuelve un valor (ej. número o booleano) se conecte a la entrada de otro.
    - **PreviousConnection:** Enlaza el bloque a una pila superior de sentencias (bloque anterior).
    - **NextConnection:** Conecta este bloque al siguiente en una secuencia lineal de sentencias (bloque inferior en la pila).
  - **InputList:** **List<Input>** que define las entradas visuales del bloque, que a su vez contienen campos o conexiones para otros bloques.

- **ChildBlocks:** `List<Block>` de los bloques lógicamente anidados (principalmente aquellos conectados a entradas de tipo sentencia o los que siguen en la ejecución secuencial vía `NextConnection`).
- **Data:** Datos adicionales asociados al bloque (ej. configuración interna de un tipo de bloque).
- **Propiedades de estado:** (`Disabled`, `Movable`, `Editable`, `IsShadow`, `Collapsed`, `InputsInline`) controlan el comportamiento y apariencia del bloque.

## Jerarquía Interna y Composición de un Bloque

La estructura de un programa se construye a partir de la jerarquía de `Blocks`. Un bloque puede ser parte de una cadena lineal de sentencias o actuar como contenedor para otros bloques hijos.

- Un **Bloque (Block)** se sitúa en la cima de su árbol jerárquico dentro del `Workspace` (un `TopBlock`), o es parte de un árbol más grande.
- Las **Conexiones Principales** (`OutputConnection`, `PreviousConnection`, `NextConnection`) vinculan el bloque a otros, definiendo su lugar en la pila o el flujo de valores.
- Los **Input** (Entradas) de un bloque actúan como puntos de acoplamiento para otros bloques (`Input Connections`) o como contenedores para **Fields** (Campos), organizando la disposición visual de los elementos internos.
- Un `Block` puede tener otros `Blocks` conectados jerárquicamente a través de sus entradas, constituyendo sus `ChildBlocks`.

## 3.4. Conexiones (Connection)

Las conexiones son el mecanismo central mediante el cual los bloques se unen entre sí, formando la estructura lógica del programa. En UBlockly, cada punto de unión se representa por una instancia de `Connection`.

### Implementación de Connection

La clase `Connection` verifica la compatibilidad entre conexiones y gestiona sus estados y relaciones. Es fundamental para el proceso de arrastre y snap (conexiones) de bloques en la interfaz.

■ **Funcionalidad principal:**

- **Verificación de compatibilidad:** Determina si dos conexiones son lógicamente compatibles según su tipo y las propiedades de `Check`. (`CanConnectWithReason()`).
- **Conexión y desconexión:** Proporciona métodos para establecer y romper la relación entre bloques (`Connect()`, `Disconnect()`).
- **Manejo de eventos:** Notifica a sus observadores (`ConnectionView`) sobre cambios de estado (`UpdateState.Connected`, `Disconnected`, `Highlight`, `BumpedAway`).
- **Lógica de Bumping:** Implementa la capacidad de mover bloques ya conectados cuando un nuevo bloque intenta unirse a una conexión ocupada, reorganizando las pilas existentes (`Connection.Connect()` y métodos auxiliares).

■ **Atributos principales:**

- **SourceBlock:** Block al que pertenece esta conexión. Crucial para la gestión de la jerarquía y la pertenencia a un `Workspace`.
- **Type:** Tipo de la conexión (`EConnection`): `InputValue`, `OutputValue`, `NextStatement`, `PrevStatement`, o `DummyInput` (cuando el Input no lleva conexión).
- **Location:** Posición *lógica* de la conexión en coordenadas del espacio de trabajo (`Vector2`), que se sincroniza con su posición visual para permitir la detección de snap entre bloques.
- **TargetConnection:** Referencia a la `Connection` a la que este bloque está unido. Es `null` si no está conectado.
- **IsConnected:** `bool` que indica si la conexión está activa.
- **Check:** `List<string>` que especifica los tipos de valores compatibles con esta conexión (e.g., `Number`, `Boolean`, `Statement`).
- **InDB:** `bool` que indica si la conexión está actualmente registrada en las bases de datos de conexión del `Workspace` (`ConnectionDBList`) para búsquedas.
- **DB / DBOpposite:** Referencias directas a las `BlockConnectionDB` correspondientes al tipo de esta conexión (`DB`) y al tipo opuesto (`DBOpposite`), para búsquedas rápidas.

### 3.5. Entradas (Input)

Un **Input** representa una entrada dentro de un bloque, ya sea para anidar otros bloques o para contener campos. Gestiona la alineación y organización de los diferentes elementos visuales dentro del bloque y su contribución al layout.

■ **Atributos principales:**

- **Name:** Nombre único de la entrada dentro de su bloque propietario (ej. STEPS, DO).
- **Type:** Tipo de la entrada (**EConnection**), indicando si es de valor (**InputValue**), de sentencia (**NextStatement**) o dummy (**DummyInput**).
- **Connection:** Si la entrada es de tipo **InputValue** o **NextStatement**, esta referencia a **Connection** permite anidar otro bloque dentro del actual.
- **FieldRow:** **List<Field>** que almacena los campos (*Fields*) o etiquetas de texto asociados a esta entrada, definiendo su apariencia interna.
- **SourceBlock:** Block padre al que pertenece esta entrada, garantizando una referencia coherente en la jerarquía del modelo.
- **ConnectedBlock:** Si **Connection** está activa, esta propiedad retorna el Block que está conectado a esta entrada.
- **Align:** Determina la alineación horizontal de los campos dentro de la entrada (**EAlign.Left**, **Center**, **Right**).

### 3.6. Campos (Field)

Los **Field** representan los datos y etiquetas individuales dentro de un bloque. Son elementos visuales editables o estáticos que muestran información relevante para la configuración del bloque.

■ **Atributos principales:**

- **Name:** Identificador único del campo dentro de su bloque.
- **Type:** Tipo del campo (**field\_label**, **field\_input**, **field\_number**, **field\_dropdown**, etc.), utilizado por la **FieldFactory** para su instanciación.

- **mText:** Valor textual actual del campo.
- **SourceBlock:** Block al que se encuentra asociado este campo.
- **IsEditable:** bool que indica si el campo permite al usuario modificar su valor en la interfaz.
- **IsImage:** bool que indica si el campo contiene una imagen en lugar de texto (ej. `field_image`).
- **Validator:** Función delegada (`FieldValidator`) que se utiliza para aplicar reglas de validación al valor de entrada del usuario, garantizando la consistencia de los datos.

### 3.7. Variables (*VariableMap*)

La clase `VariableMap` es el componente del `Workspace` que gestiona la creación, eliminación, renombrado y almacenamiento de las variables que los usuarios definen en el entorno de programación.

■ **Atributos clave:**

- **mVariableMap:** `Dictionary<string, List<Variable>` que agrupa las variables por su tipo (cadena, número, etc.), permitiendo un acceso y una gestión eficientes.
- **mWorkspace:** Hace referencia a la instancia del `Workspace` asociada, garantizando la pertenencia y el contexto operativo del mapa de variables.

### 3.8. Sistema de ejecución (*CSharpRunner* e *Cmdtors*)

El corazón de la lógica programable reside en el sistema de ejecución, responsable de interpretar y ejecutar la secuencia de bloques del usuario. En el prototipo, esto se realiza mediante un intérprete directo basado en corrutinas de *C#*, orquestado por *CSharpRunner* y los intérpretes de bloques (*Cmdtors*).

#### Clases del sistema de ejecución

- **Cmdtor:** Clase base abstracta para todos los intérpretes de bloques. Cada *Cmdtor* (o sus subclases: *ValueCmdtor*, *VoidCmdtor*, *EnumeratorCmdtor*)

define cómo un tipo de bloque específico traduce su lógica a acciones programáticas. En el prototipo, **MotionBlockInterpreter** es un ejemplo concreto de esta implementación para el bloque de movimiento.

- **CmdEnumerator:** Clase auxiliar que envuelve un **IEnumerator** de un **Cmdtor**, permitiendo que el **CSharpRunner** lo gestione como una corrutina en una pila. Facilita la ejecución secuencial y la pausa/reanudar de las operaciones de los bloques.
- **CSharpRunner:** Componente **MonoBehaviour** que orquesta la ejecución del programa. Mantiene una pila de corrutinas (**Stack<IEnumerator>**) para manejar el flujo de ejecución (secuencial, anidado, condicionales, bucles) de los bloques. Permite el control del ciclo de vida del programa (inicio, pausa, reanudación, parada y modo paso a paso). El **CSharpRunner** (a través del **CmdRunner**) gestiona las corrutinas de los **Cmdtor**. El **BlockObserver** es un componente que los **Cmdtor** (como **MotionBlockInterpreter**) pueden usar para interactuar con el mundo de Unity (el robot)

### 3.9. Representación visual y composición de bloques en Unity

Para adaptar UBlockly a Unity, cada **Block** (la representación lógica de un bloque) se traduce en un **GameObject** con componentes **UnityEngine.UI** que conforman su *vista*. La **BlockView (BaseView)** es el componente principal que gestiona esta representación visual y sincroniza el modelo lógico con la interfaz de usuario. Este proceso de adaptación busca replicar la apariencia y el comportamiento de los bloques de Scratch, combinando la flexibilidad de UBlockly con una estética intuitiva.

#### Creación de prefabs para bloques en Unity

En Unity, un **Prefab** es un tipo de activo que permite almacenar un **GameObject** con sus componentes, propiedades y toda su jerarquía de hijos como una plantilla reutilizable. Esta aproximación facilita la creación consistente y el mantenimiento de múltiples instancias de un mismo objeto en una escena o a través del tiempo. UBlockly ofrece métodos tanto para generar los bloques de manera procedimental en tiempo de ejecución como para utilizar prefabs prediseñados. Para el desarrollo de este prototipo, se ha optado por un enfoque que combina el uso de prefabs configurados manualmente en el

editor de Unity con la flexibilidad de la estructura lógica de UBlockly. Esta decisión permite un control granular sobre la apariencia y el comportamiento visual exacto de cada bloque, asegurando que el prototipo se ajuste a la experiencia visual deseada de Scratch. Cada tipo de bloque (como, por ejemplo, mover X pasos) se ha diseñado como un prefab distinto, preconfigurado con su estructura visual base y componentes de `UnityEngine.UI`.

## Mapecto Modelo-Vista y elementos UI

La sincronización entre la lógica de programación (el Modelo, `Block`) y su presentación visual (la Vista, `BlockView`) es fundamental. Esta relación bidireccional se mantiene mediante un patrón Observador, donde los cambios en el modelo (`Block.FireUpdate()`) notifican a la vista (`BlockView.HandleModelUpdate()`) para que se actualice. A su vez, las interacciones del usuario en la vista son traducidas por los controladores para modificar el modelo.

- **Elementos UI comunes:** Los bloques se componen visualmente de una combinación de los siguientes componentes Unity UI, cada uno asociado a una clase `View` específica:
  - **Image:** Utilizado para la forma base del bloque, su color y sus muescas/pestañas características. Se utiliza un componente especializado (`CustomMeshImage`) que permite la manipulación de la geometría del bloque para adaptarla a la forma de las muescas, pestañas y conexiones de Scratch.
  - **TextMeshProUGUI:** Representan las etiquetas de texto de los bloques (ej. mover, pasos) y los valores numéricos o de texto editables por el usuario.
  - **Button:** Utilizado para elementos interactivos como campos de selección desplegados (`TMP_Dropdown` para `FieldDropdownView` y `FieldVariableView`) y posibles botones en el futuro (`FieldButtonView`).

## Sistema de posicionamiento y tamaño (Layout Manual)

El correcto despliegue visual de los bloques en la interfaz requiere un sistema de layout preciso, dado que Unity UI no siempre ofrece un ajuste automático óptimo para las formas complejas y anidadas de los bloques de programación visual.

- **RectTransform, Anchor points y Pivot:** Todos los elementos UI (`GameObjects`) de los bloques poseen un componente `RectTransform`. Su posicionamiento y redimensionamiento se gestionan a través de la manipulación programática de sus propiedades `anchoredPosition`, `pivot` y `anchor` en un **sistema de layout manual**. Este sistema calcula dinámicamente el tamaño (`CalculateSize()`) y la posición (XY) de cada `BlockView` y sus hijos (`InputView`, `FieldView`, `ConnectionView`) en la jerarquía visual de la UI.
- **Conexiones Visuales (`ConnectionView`):** Cada `Connection` tiene una `ConnectionView` asociada que representa visualmente el punto de conexión. Estas vistas gestionan el *resaltado visual* (`Highlight`) para indicar posibles conexiones durante el arrastre, y son clave en el *re-parentado y posicionamiento de bloques* tras un snap. Aunque no se emplean `LineRenderer` o `Bezier Curves` para el trazado de las conexiones (como ocurre en otras herramientas de modelado de software), el snap visual y la animación son fundamentales para la experiencia de usuario.

### 3.10. Serialización y persistencia

Para permitir guardar y cargar el estado de los programas del usuario, el prototipo incorpora un sistema de serialización y persistencia. Este sistema se encarga de almacenar la estructura lógica del `Workspace` y de restaurarla cuando sea necesario, permitiendo la continuidad en los proyectos.

- **Formato de datos:** La estructura de los bloques y sus relaciones se almacenan en formato **XML**, el cual permite una representación jerárquica clara del programa.
- **Clases involucradas:**
  - **Xml:** Componente del core de UBlockly responsable de transformar la estructura del `Workspace` a un árbol `XMLDocument` (`WorkspaceToDom()`) y viceversa (`DomToWorkspace()`).

### 3.11. Arquitectura del Prototipo Final: Un Sistema Híbrido

Tras analizar la complejidad inherente a la implementación de un motor de layout visual completo, se tomó la decisión estratégica de pivotar el



desarrollo hacia un Producto Mínimo Viable (MVP) que mantuviera la claridad de un sistema basado en bloques pero introdujera la escritura de código. Este enfoque híbrido aprovecha la base conceptual del MVC para desacoplar un nuevo sistema de programación textual de la simulación del robot en el entorno 3D.

## Componentes del sistema híbrido

La nueva arquitectura se compone de los siguientes elementos clave:

- **Gestor de desafíos (`ChallengeLoader`, `ChallengeData`):** Sistema basado en Scriptable Objects para definir y cargar los diferentes escenarios de aprendizaje, desacoplando el contenido de la lógica del programa.
- **Interfaz de desarrollo (IDE - `IDE_UIManager`):** Es la Vista principal para el usuario en la escena de programación. Gestiona la presentación de los objetivos del desafío, el área de entrada de código y la visualización del programa como una secuencia de instrucciones textuales. Actúa como el puente entre el usuario y el `ProgramBuilder`.
- **Constructor del programa (`ProgramBuilder`):** Acumula las líneas de código introducidas por el usuario, actuando como un búfer o editor de texto simple. Al ejecutar, ensambla el código fuente completo.
- **Analizador Sintáctico (`CommandParser`):** Este componente es el núcleo del intérprete del DSL. Su función es recibir el código fuente, tokenizarlo (dividirlo en comandos y argumentos) y validarlo sintácticamente. Si el código es válido, genera una estructura de datos intermedia: una lista de objetos `Instruction`, que es el análogo a un Árbol de Sintaxis Abstracto (AST) en un compilador tradicional.
- **Intérprete y motor de simulación (`GameManager`, `RobotController`):** Reciben la lista de instrucciones y la ejecutan de manera secuencial, traduciendo cada `Instruction` en una acción concreta del robot (ej. una corrutina de movimiento o rotación). Este módulo gestiona el estado de la simulación y detecta eventos como las colisiones.

### 3.12. Objetivos de Desarrollo Sostenible (ODS) y su Aplicación en el Proyecto

Los Objetivos de Desarrollo Sostenible (ODS) son una lista de 17 objetivos globales adoptados por la Asamblea General de la Organización de las Naciones Unidas (ONU) en 2015, como parte de la Agenda 2030 para el Desarrollo Sostenible. Estos objetivos universales e interconectados buscan abordar desafíos globales urgentes, como la pobreza, la desigualdad, el cambio climático, la degradación ambiental, la paz y la justicia. Su integración en proyectos de ingeniería informática refleja un compromiso con la creación de soluciones con un impacto positivo a nivel social, económico y ambiental.

El presente TFG se alinea con los ODS al fomentar una educación inclusiva y al reducir las desigualdades, mediante la democratización del acceso a herramientas de aprendizaje tecnológico.

#### ODS 4: Educación de Calidad

El **ODS 4: Educación de Calidad** busca Garantizar una educación inclusiva y equitativa de calidad y promover oportunidades de aprendizaje permanente para todos[21]. En el contexto de este proyecto, se contribuye a este ODS de las siguientes maneras:

- **Acceso equitativo:** El laboratorio virtual reduce significativamente las barreras de entrada a la educación en programación y robótica. Al eliminar la necesidad de hardware físico costoso y laboratorios especializados, el proyecto democratiza el acceso, permitiendo a estudiantes de zonas con recursos limitados o sin acceso a infraestructuras dedicadas, participar en experiencias de aprendizaje enriquecedoras.
- **Educación inclusiva:** Al ofrecer una interfaz basada en la programación por bloques tipo Scratch, accesible e intuitiva, se facilita el aprendizaje a niños y adolescentes con diversos niveles de experiencia y habilidades. El diseño del entorno está concebido para ser comprensible para un público amplio, desde principiantes hasta usuarios más avanzados.
- **Aprendizaje permanente:** La adquisición de habilidades en pensamiento computacional y lógico desde edades tempranas sienta una base sólida para el aprendizaje continuo en el ámbito de las tecnologías.

De esta forma, el proyecto es una herramienta que promueve activamente la participación de una comunidad estudiantil más amplia en las disciplinas STEM [21].

## ODS 10: Reducción de las Desigualdades

El **ODS 10: Reducción de las Desigualdades** tiene como meta Reducir la desigualdad en los países y entre ellos [21]. La contribución del proyecto se manifiesta en los siguientes aspectos:

- **Democratización del acceso a la tecnología educativa:** El coste de la tecnología educativa es un factor determinante que a menudo excluye a segmentos de la población. Al priorizar el desarrollo de una aplicación para dispositivos Android (como tablets de bajo coste) y utilizar software de código abierto (Unity Personal, C#, UBlockly), el proyecto minimiza la inversión económica requerida para acceder a estas herramientas didácticas.
- **Brecha digital y geográfica:** En regiones donde la infraestructura tecnológica y los recursos educativos son escasos, las simulaciones virtuales pueden compensar la falta de laboratorios físicos y equipamiento de robótica. Esto reduce la brecha digital y geográfica, permitiendo que estudiantes de entornos rurales o menos privilegiados accedan a la misma calidad de experiencia de aprendizaje que sus homólogos en áreas urbanas.

Este proyecto no solo facilita el aprendizaje técnico, sino que también sirve como una herramienta para fomentar una sociedad más justa e igualitaria al hacer la educación en programación y robótica más accesible a todos.



---

## 4. Técnicas y herramientas

---

El presente capítulo tiene como objetivo describir las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Se detallarán los aspectos más destacados de cada elección, justificando el porqué de su utilización frente a posibles alternativas, y cómo estas herramientas han contribuido al logro de los objetivos planteados. Para mejorar la comprensión, las herramientas se han agrupado en subsecciones según su área de aplicación.

### 4.1. Metodología de desarrollo: Agile con SCRUM

El desarrollo del prototipo de laboratorio virtual se ha abordado mediante la aplicación de principios de metodologías ágiles, en particular el marco de trabajo **SCRUM**. Este enfoque, a diferencia de modelos lineales o en cascada, ha sido fundamental por su énfasis en la flexibilidad, la colaboración y la entrega continua de valor. Permite una rápida adaptación a los desafíos técnicos y a la evolución de los requisitos durante el desarrollo de un proyecto complejo y en evolución como este.

- **Planificación iterativa (Sprints):** El desarrollo se ha estructurado en *sprints* de duración fija (ej. 1-3 semanas), al final de los cuales se obtenía un incremento funcional y probado del prototipo.
- **Adaptación y pivote estratégico:** La naturaleza ágil de SCRUM fue clave para gestionar la evolución del proyecto. Tras evaluar la complejidad de implementar un motor de layout visual completo,

la metodología permitió pivotar estratégicamente hacia un modelo híbrido. Este cambio se gestionó como una redefinición del alcance dentro de un sprint, asegurando la entrega de un Producto Mínimo Viable (MVP) sin comprometer los plazos ni la calidad del núcleo funcional

- **Roles y colaboración:** El rol de tutor (D. Carlos López Nozal) ha adoptado las funciones de *Product Owner* (priorizando funcionalidades y validando incrementos) y *Scrum Master* (guiando la aplicación de la metodología). El rol del alumno (autor del TFG) ha sido el de *Equipo de Desarrollo*.
- **Monitorización del progreso:** La plataforma **Zube** [25] ha sido utilizada para la gestión de proyectos, integrándose con GitHub y permitiendo el uso de tableros Kanban para planificar tareas, asignar puntos de historia y realizar seguimiento visual del progreso.
- **Integración continua y calidad del código:** Como parte del compromiso con la calidad del software, se han explorado prácticas de integración continua (CI), aunque no completamente automatizadas, con revisiones periódicas del código en GitHub y una inspección básica de las estructuras y coherencia implementadas, detectando posibles errores y problemas de estilo o rendimiento.

La adopción de SCRUM no solo ha facilitado la gestión del proyecto, sino que ha sido el pilar que ha permitido una toma de decisiones ágil ante desafíos técnicos, reorientando el esfuerzo hacia la creación de un MVP robusto y funcional.

## 4.2. Herramientas core para el desarrollo de software

### Motor de videojuegos: Unity 6 y Lenguaje C#

El corazón del laboratorio virtual reside en el motor de videojuegos **Unity Versión: 6000.0.26f1**, elegido por su potente capacidad para la creación de entornos 3D interactivos y simulaciones en tiempo real. La decisión de utilizar Unity, frente a motores exclusivamente de renderizado, se fundamenta en sus capacidades para:

- **Simulación 3D avanzada:** Permite el modelado, animación y simulación de la física del robot, esenciales para replicar comportamientos realistas.
- **Sistema de UI unificado (Unity UI Canvas):** Proporciona un marco robusto para construir interfaces de usuario complejas mediante un **Unity UI Canvas** (**UICanvasView**), lo que ha facilitado la creación del editor de bloques con su interacción visual.
- **Compatibilidad multiplataforma:** Soporta la exportación a diversos sistemas operativos, incluyendo **Android (Tablets)**, que es la plataforma objetivo del prototipo, lo cual es crítico para la accesibilidad del proyecto.
- **Comunidad y recursos extensivos:** Dispone de una vasta comunidad de desarrolladores y una amplia biblioteca de recursos que agilizan el desarrollo.

El lenguaje de programación principal utilizado es **C#**. La integración nativa de **C#** con Unity permite el desarrollo de scripts eficientes basados en programación orientada a objetos (POO), y el uso intensivo de **corrutinas** (**IEnumerator**) para la gestión de la lógica asíncrona de la ejecución de bloques y animaciones, crucial para una experiencia fluida del usuario. El entorno de desarrollo integrado (IDE) principal ha sido **Visual Studio**.

## Modelado 3D y creación de modelos de robot

Para la creación del modelo 3D del robot y la representación de componentes específicos de LEGO WeDo 2.0, se han utilizado herramientas de modelado dedicadas:

- **Blender (Versión 4.4):** Como software de modelado 3D principal. Blender permitió diseñar un modelo de robot de forma personalizada, optimizar la malla para asegurar un rendimiento eficiente en tiempo real en Unity, y exportar los modelos en formatos compatibles.
- **LEGO Digital Designer (LDD) o Studio 2.0:** Como herramientas auxiliares, han sido útiles para entender la composición de los robots LEGO WeDo 2.0 y para generar ideas de construcción de modelos 3D que repliquen su estética, sirviendo como referencia para el modelado en Blender.

Esta fase ha sido fundamental para transformar la idea de un robot LEGO WeDo 2.0 virtual en un activo 3D funcional dentro de Unity, que sirve como base para la simulación, con una estética inspirada en robots educativos como LEGO WeDo 2.0, asegurando la cohesión visual del proyecto.

## Programación visual y textual: UBlockly y DSL propio

- **Base lógica para programación por bloques:** UBlockly proporcionó una estructura de referencia para la representación lógica de bloques (**Block**) y la gestión de sus conexiones (**Connection**). Esta base fue utilizada para implementar los bloques estructurales del sistema, como el bloque de evento inicial y los bucles, acelerando el desarrollo de la parte visual.
- **Desarrollo de un lenguaje de dominio específico (DSL):** Para los comandos de acción del robot (movimiento y giro), se optó por diseñar e implementar un DSL textual. Esta decisión permitió un control más directo sobre la semántica de las acciones del robot y simplificó la ejecución, eliminando la necesidad de un motor de layout visual complejo para estos comandos. La implementación incluyó:
  - Un Parser (**CommandParser**) que analiza el código escrito por el usuario.
  - Un Intérprete (**RobotController**) que ejecuta la secuencia de instrucciones resultante.
- **Implementación híbrida:** La combinación de bloques visuales para la estructura y código textual para las acciones conforma el núcleo híbrido del prototipo. Esta aproximación no solo es una solución pragmática a los desafíos técnicos, sino que también ofrece un modelo pedagógico de transición, donde los usuarios aprenden la estructura del programa visualmente y practican la codificación de comandos de forma escrita.

### 4.3. Persistencia de datos del programa

Para permitir guardar y cargar los programas creados por el usuario, el prototipo incorpora un sistema de persistencia básico.



## Sistema de gestión de Desafíos y guardado de código

La persistencia de datos en el prototipo se aborda de dos maneras complementarias:

- **Gestión de desafíos con Scriptable Objects:** La definición de los desafíos (título, descripción, etc.) se gestiona mediante **Scriptable Objects** (ChallengeData). Este enfoque de Unity permite desacoplar los datos del contenido de la lógica del juego, facilitando la creación y modificación de nuevos desafíos sin necesidad de cambiar el código. Un gestor (ChallengeLoader) se encarga de la persistencia de la selección del usuario entre escenas.
- **Persistencia del programa de usuario (No implementada en el MVP):** Aunque la arquitectura de UBlockly analizada incluía un sistema de serialización a XML, en la versión actual del prototipo no se ha implementado una funcionalidad de guardado/carga del programa creado por el usuario. La lógica de los desafíos está diseñada para ser resuelta en una sola sesión, enfocándose en la resolución de problemas y la ejecución inmediata del código. La implementación de la persistencia se considera una línea de trabajo futura.

## 4.4. Herramientas para gestión de proyectos y desarrollo

### Control de versiones: Git y GitHub

El control de versiones es fundamental para el desarrollo colaborativo y el seguimiento de los cambios en el código. **Git** ha sido utilizado como sistema de control de versiones distribuido, permitiendo un registro detallado del historial del proyecto y la gestión eficiente de las distintas ramas de desarrollo. El repositorio en **GitHub** ha servido como plataforma de alojamiento remoto, facilitando la colaboración, el versionado y la sincronización del código fuente. Sus características, como la gestión de *\*issues\** y *\*milestones\** han sido valiosas para estructurar y monitorizar el avance del proyecto.

### Asistencia en codificación: GitHub Copilot

Durante la fase de implementación, se ha hecho uso de **GitHub Copilot**, una herramienta de asistencia en la codificación basada en Inteligencia Artificial [8]. Copilot, que se integra con IDEs como Visual Studio, ofrece

sugerencias de código en tiempo real, completado automático y la generación de fragmentos de código basados en el contexto. Esto ha contribuido a aumentar la eficiencia del desarrollo, reduciendo la necesidad de escribir código repetitivo y facilitando la exploración de patrones de programación. Su aplicación se ha limitado a tareas de apoyo, manteniendo el control total del código en manos del desarrollador.

## **Edición y compilación de documentación: TeXstudio y LaTeX**

La documentación de la memoria del Trabajo Fin de Grado ha sido elaborada utilizando **LaTeX** como sistema de composición de textos, conocido por su capacidad para generar documentos de alta calidad tipográfica y su robustez en la gestión de referencias bibliográficas. La herramienta **TeXstudio** ha actuado como Entorno de Desarrollo Integrado (IDE) para LaTeX, facilitando la escritura, la corrección ortográfica y gramatical, el resaltado de sintaxis y la previsualización en tiempo real del documento. Esta combinación ha sido esencial para la producción de una memoria técnica profesional. .

---

## 5. Aspectos relevantes del desarrollo del proyecto

---

Este capítulo tiene como objetivo recoger los aspectos más interesantes y no triviales del proceso de desarrollo del prototipo del laboratorio virtual de programación y robótica. Se detallarán las decisiones de diseño arquitectónico y de implementación clave, justificando los caminos adoptados y las estrategias empleadas para resolver los desafíos técnicos, desde la concepción inicial de un sistema de programación visual hasta la implementación final de una solución híbrida. Se busca que este apartado sirva como un resumen de la experiencia práctica del proyecto y una referencia útil para futuros desarrolladores.

### 5.1. Metodología de desarrollo: Iteraciones ágiles con SCRUM

El proceso de desarrollo de este prototipo ha seguido una metodología adaptativa, inspirada en los principios de **SCRUM**. Esta elección, frente a modelos más lineales, se justificó por la naturaleza experimental del proyecto y la necesidad de una *respuesta rápida* a los desafíos tecnológicos emergentes. La aplicación de este marco ágil permitió:

- **Entrega de valor incremental:** El proyecto se estructuró en ciclos de *sprints* de duración fija (ej., una a tres semanas), donde al final de cada iteración se obtenía un incremento funcional y probado del prototipo. Esto aseguró un progreso tangible y la validación temprana

de funcionalidades críticas, como el movimiento del robot, que era la piedra angular del prototipo.

- **Flexibilidad y adaptación continua:** La revisión constante del progreso con el tutor (asumiendo roles de *Product Owner* y *Scrum Master*) permitió reevaluar prioridades, adaptar requisitos y refinar las soluciones técnicas en función de la experiencia práctica obtenida, así como gestionar la complejidad que surgió de la interacción entre UBlockly y Unity.
- **Transparencia y gestión del trabajo:** Herramientas como Zube [25] se emplearon para la planificación de *sprints*, el seguimiento de tareas con tableros *Kanban* y la asignación de *puntos de historia*. Esto facilitó la visualización del trabajo pendiente y realizado, mejorando la organización del proyecto siendo acompañado de comentarios, capturas de imágenes de la aplicación y videos donde se observaba la evolución del trabajo llevado a cabo para una correcta retroalimentación con el Tutor del TFG.

Este enfoque iterativo fue clave para la gestión eficaz de la complejidad técnica del proyecto y para alcanzar un *Producto mínimo viable (MVP)* dentro del plazo establecido.

## 5.2. Diseño e implementación de la arquitectura inicial

El punto de partida del proyecto fue el diseño de una arquitectura robusta para un sistema de programación puramente visual, tomando como referencia técnica la librería UBlockly [9]. A continuación, se detallan las decisiones clave de diseño tomadas en esta fase inicial, que sentaron las bases para todo el desarrollo posterior.

### Adopción y mejora de UBlockly: Adaptación a MVC

Tras un análisis de la librería UBlockly, se identificó que, si bien proporcionaba un modelo de datos sólido, carecía de una separación estricta de responsabilidades entre la lógica y la representación visual. Como primera y fundamental aportación, se refactorizó su estructura para adaptarla al patrón arquitectónico Modelo-Vista-Controlador (MVC). Esta decisión fue crucial para gestionar la complejidad y sentar las bases de un sistema mantenible y escalable.

- **Modelo:** Las clases centrales del Modelo (`WorkSpace`, `Block`, `Connection`, `Input`, `Field`, `VariableMap`, `ProcedureDB`) encapsulan el estado lógico y las reglas de negocio del sistema de bloques. Son independientes de cualquier representación visual y se enfocan en la *abstracción de la programación visual* del usuario.
- **Vista (Representación visual e interacción UI):** Componentes Unity UI como `UnityEngine.UI.Image`, `TMPPro.TextMeshProUGUI`, `UnityEngine.UI.Button` actúan como los elementos visuales, gestionados por una jerarquía de clases `View` (`BaseView`, `BlockView`, `ConnectionView`, `InputView`, `FieldView`, `LineGroupView`). Estas vistas son las únicas que conocen la interfaz de Unity y se encargan de representar el Modelo.
- **Controlador (Orquestación e interpretación de interacciones):** Capas de lógica como `WorkspaceController`, `BlockController`, `TollboxController`, median entre el Modelo y la Vista. Interpretan las acciones del usuario, validan operaciones y orquestan las actualizaciones del Modelo y la Vista sin que estas capas se conozcan directamente entre sí.

La comunicación entre estas capas se gestiona primordialmente mediante el **patrón Observador** (`Observable<TArgs>` e `IObserver<TArgs>`). Los modelos (`Block`, `Connection`, `Field`) actúan como sujetos observados, notificando de cualquier cambio relevante a sus vistas o controladores suscritos. Esto asegura una *sincronización eficiente* de la interfaz sin acoplamientos rígidos entre los elementos.

## Sistema de programación visual: Componentes clave

La decisión de basar la implementación del motor de bloques en el repositorio de **UBlockly** [9], una adaptación de Blockly en C# para Unity, fue una *elección estratégica* frente al desarrollo desde cero. Este enfoque se justificó por la necesidad de construir un prototipo funcional robusto en un tiempo limitado, ya que UBlockly proporcionó una base sólida para:

- **Motor lógico de bloques:** Sus clases core como `WorkSpace` (gestor principal de la base de datos de bloques), `Block` (estructura lógica de un bloque) y `Connection` (sistema de conexiones entre bloques) ofrecieron una base abstracta y funcional ya testada. Se adoptó este diseño tal cual, centrándose la adaptación en cómo estos modelos interactúan con la nueva capa de vistas en Unity.

- **Motor de ejecución asíncrono:** UBlockly ya incluye un robusto sistema de interpretación directa basado en `Cmdtors` y corrutinas de C# (`CSharpRunner`). Esta arquitectura se ha adoptado directamente, adaptando los intérpretes (`MotionBlockInterpreter`) para interactuar con los objetos simulados del robot.
- **Serialización y persistencia:** El mecanismo de UBlockly para serializar el estado del `Workspace` a XML y viceversa se integró directamente, proporcionando una solución eficiente para guardar y cargar los proyectos del usuario.

La adaptación principal ha consistido en *acoplar las capas de modelo y ejecución de UBlockly con la nueva capa de visualización de Unity UI*, así como *customizar la lógica de interacción* (drag & drop) para lograr una experiencia más cercana a Scratch. Se han creado prefabs de `GameObjects` de Unity con la estructura de componentes `View` que reflejan los `Blocks` lógicos, permitiendo la creación modular y una apariencia flexible de los bloques.

### 5.3. Detalles clave de implementación

#### Representación visual y composición de bloques en Unity

Uno de los mayores desafíos en el desarrollo de interfaces de programación visual es la correcta representación y sincronización entre el modelo lógico del programa y su visualización interactiva. En este proyecto, la aproximación se basa en la creación de **prefabs** de `GameObjects` de Unity para cada tipo de bloque (`BlockView`), replicando la estética de Scratch.

- **Creación de prefabs estructurados:** Cada `BlockView` es un prefab con una jerarquía de componentes `UnityEngine.UI` que se corresponden directamente con las estructuras lógicas del `Block` (ej., `InputView` para entradas, `FieldView` para campos). Esta composición predefinida de prefabs acelera la instanciación de bloques en tiempo de ejecución.
- **Layout manual y sincronización de `RectTransform`:** A diferencia de los sistemas de layout automático de Unity, el proyecto emplea un **sistema de layout manual** (`BaseView.UpdateLayout()`) para los bloques. Este sistema calcula dinámicamente la posición (XY) y

el tamaño (`CalculateSize()`) de cada `BlockView` y sus subcomponentes (`InputView`, `FieldView`, `ConnectionView`). Es crucial porque los bloques de programación visual tienen formas complejas (muescas, pestañas, huecos para entradas) que no se ajustan de manera óptima a layouts grid o verticales/horizontales simples. Cada vez que el modelo lógico de un bloque cambia de posición (`BlockModel.XY`), o su contenido o tamaño (`Block.FireUpdate()` para `UpdateStates.Inputs` o `UpdateStates.Connections`), el `BlockView` es notificado y marca su layout como sucio (`NotifyLayoutDirty`). Posteriormente, en un ciclo de Unity, se fuerza un recálculo del layout completo o parcial mediante `LayoutRebuilder.ForceRebuildLayoutImmediate()`.

- **Visualización de conexiones e identificación de `nextStatementContainer`:** Las conexiones lógicas (`Connection`) tienen su reflejo visual en las `ConnectionViews`.

## El Motor de ejecución de bloques: De la lógica a la acción del robot

El corazón operativo del prototipo es el motor de ejecución de bloques, responsable de interpretar la secuencia de instrucciones del usuario y traducirlas en acciones en el entorno de simulación 3D.

- **Intérpretes de bloques (`Cmdtors`):** Cada tipo de bloque funcional (ej. movimiento) tiene una clase intérprete concreta que hereda de `Cmdtor` (`ValueCmdtor`, `VoidCmdtor`, `EnumeratorCmdtor`). Por ejemplo, `MotionBlockInterpreter` se encarga de extraer los parámetros de mover N pasos de un `BlockModel`.
- **Orquestación con `CSharpRunner`:** La clase `CSharpRunner`, un `MonoBehaviour`, es el orquestador principal de la ejecución. Mantiene una `Stack<IEnumerator>` de `CmdEnumerators` para gestionar el flujo de control (secuencia lineal, bucles, condicionales). Esto permite una *ejecución paso a paso no bloqueante* del programa.
- **Concurrencia con corrutinas:** La característica distintiva de este sistema es el uso intensivo de las **corrutinas de C# (`IEnumerator`, `yield return`)**. Cuando un intérprete de bloque (`Cmdtor`) invoca una acción que toma tiempo (ej., una animación de movimiento del robot, un retardo), este se traduce en un `yield return` en C#. El `CSharpRunner` *pausa la ejecución de ese bloque* y la *reanuda automáticamente* cuando la corrutina de la acción externa ha finalizado. Esto es

crucial para lograr *animaciones fluidas* del robot o *retrasos precisos* sin congelar la aplicación. Por ejemplo, `MotionBlockInterpreter` llama a `BlockObserver._MoveRobot()` (una corrutina), y el `CSharpRunner` espera a que esta corrutina termine antes de pasar al siguiente bloque.

- **Sincronización con la Simulación:** La clase `BlockObserver` actúa como un puente entre el motor de ejecución de bloques y el `GameObject` del robot simulado (`RobotBehaviour`). `BlockObserver` recibe las órdenes de alto nivel (ej. mover 10 pasos) desde los `Cmdtors` y delega estas acciones a `RobotBehaviour` (el script que controla el movimiento y la animación física del robot), asegurando que la simulación refleje con precisión la lógica del programa. `BlockObserver` también gestiona el feedback visual en la interfaz de usuario, como los mensajes de estado y el resaltado del bloque en ejecución.

## 5.4. Pivote estratégico y diseño del sistema híbrido final

Tras implementar y evaluar la arquitectura inicial, se identificó que la creación y mantenimiento de prefabs de `BlockView` y el perfeccionamiento del motor de layout dinámico para todos los bloques necesarios suponían un esfuerzo de desarrollo que superaba el alcance del proyecto. Con el objetivo de entregar un producto educativo funcional, se tomó la decisión estratégica de pivotar a un sistema que combinara una representación visual simplificada con un sistema de programación textual dirigido.

### Componentes clave de la arquitectura híbrida

La arquitectura del prototipo final, aunque inspirada en los principios de separación de responsabilidades, se simplificó para adaptarse al nuevo enfoque híbrido. La interacción entre la entrada del usuario y la simulación del robot está orquestada por un conjunto de gestores especializados:

- **GameManager:** Actúa como el controlador central de la escena de programación. Es responsable de recibir el programa parseado desde la interfaz de usuario y de iniciar la ejecución en el `RobotController`. Además, gestiona el estado general del desafío, como la finalización de la ejecución o la detección de eventos críticos (colisiones).
- **IDE\_UIManager:** Este componente gestiona toda la interfaz de usuario de la escena de codificación. Captura las entradas del usuario (nuevos



comandos de texto, clics en los botones ejecutar o borrar) y las comunica al **GameManager**. También se encarga de actualizar la UI con el feedback del sistema, como los mensajes de error del parser o los mensajes de estado de la ejecución. Es la Vista-Controlador principal de la interacción del usuario.

- **RobotController:** Aunque principalmente es parte del Modelo de simulación (define el estado y comportamiento del robot), también actúa como un controlador de bajo nivel para sus propias acciones. Contiene la lógica de ejecución de las instrucciones ('Instruction') individuales en forma de corrutinas, manejando directamente la física (**Rigidbody**) y la cinemática del robot.
- **CommandParser:** Si bien no es un controlador en el sentido tradicional de MVC, esta clase estática implementa la lógica de negocio fundamental para la traducción del código fuente a una estructura de datos ejecutable. Es una pieza clave que desacopla la entrada de texto del motor de ejecución del robot.
- **ChallengeLoader:** Funciona como un controlador de datos entre escenas. Utilizando el patrón Singleton, se encarga de persistir la selección del desafío del usuario desde la escena del menú y de proporcionar esta información al **GameManager** al iniciar la escena de programación.

Esta estructura, aunque más directa que un sistema MVC de bloques completo, mantiene una clara separación de responsabilidades: la UI gestiona la entrada/salida, el GameManager orquesta el flujo, el Parser traduce la lógica, y el RobotController ejecuta las acciones físicas.

## Diseño del sistema de desafíos y del DSL

La implementación final se basa en los siguientes componentes:

- **Gestión de desafíos con Scriptable Objects (ChallengeData):** Para estructurar el contenido educativo de forma modular y escalable, se utilizó el sistema de Scriptable Object' de Unity. Cada ChallengeData define un desafío con su título, descripción e instrucciones, permitiendo una fácil creación de nuevo contenido sin modificar el código. Un gestor singleton (ChallengeLoader) asegura la persistencia de la selección del desafío entre la escena del menú y la de programación.

- **La interfaz de desarrollo (IDE - IDE\_UIManager):** Se implementó una interfaz de usuario específica para la resolución de los desafíos. Esta vista principal gestiona:
  - La presentación de la información del ChallengeData activo.
  - Un campo de entrada (`singleCommandInput`) para que el usuario escriba los comandos del DSL.
  - La visualización del programa como una lista de bloques de texto instanciados desde un prefab (`commandBlockPrefab`), proporcionando un feedback visual inmediato de la secuencia de comandos.
  - La comunicación de las acciones del usuario (ejecutar, limpiar) al `GameManager`.
- **Implementación del intérprete (Parser y Ejecutor):** En el núcleo del sistema se encuentra un intérprete para un Lenguaje de Dominio Específico (DSL) diseñado a medida:
  - **Análisis léxico y sintáctico (`CommandParser`):** Este parser, aunque simple, realiza las funciones básicas de un compilador. Recibe el código fuente como una cadena de texto, lo divide en líneas y tokens (palabras). Mediante una estructura switch-case, identifica los comandos (mover, girar, repetir), valida sus argumentos y sintaxis, y en caso de éxito, traduce cada línea a un objeto `Instruction`. El conjunto de estas instrucciones forma una representación intermedia del programa, análoga a un árbol de sintaxis abstracto (AST) simple.
  - **Manejo de estructuras anidadas:** El parser utiliza una pila (Stack) para gestionar correctamente los bloques de bucle repetir ... fin\_repetir, asegurando que las instrucciones se aniden jerárquicamente en la estructura de datos final.

## Implementación de la simulación y ejecución del robot

El `GameManager` actúa como el orquestador principal en la escena de simulación, mientras que el `RobotController` encapsula la lógica y física del robot.

- **Motor de ejecución de instrucciones:** El `RobotController` recibe la `List<Instruction>` generada por el parser. Inicia una corrutina principal (`ProcessInstructionList`) que itera sobre la lista. Por cada

instrucción, invoca una sub-corrutina específica (e.g., `MoveForwardRoutine`). El uso de `yield return` en estas llamadas asegura que una instrucción no comience hasta que la anterior haya finalizado, garantizando una ejecución secuencial predecible.

- **Física y detección de colisiones:** El robot utiliza un componente `Rigidbody` para interactuar con el entorno. Para la detección de colisiones, se implementó una estrategia proactiva mediante `Rigidbody.SweepTest`. Antes de cada movimiento, se realiza un barrido en la dirección del desplazamiento para prever si habrá una colisión. Si se detecta un obstáculo, el movimiento se limita a la distancia hasta el punto de impacto y se notifica al `GameManager` para que detenga la ejecución. Esto proporciona un feedback inmediato y realista al usuario.

## 5.5. Retos técnicos abordados y estrategias de depuración

El desarrollo de un sistema híbrido de esta complejidad, integrando una librería de lógica de bloques con el entorno 3D y UI de Unity, ha presentado desafíos técnicos significativos.

- **Sincronización de modelos y vistas en MVC con corrutinas:** El reto de mantener el `Block` lógico (con posiciones `Vector2` abstractas) perfectamente sincronizado con el `RectTransform` visual en el `Unity UI Canvas` ha requerido una depuración exhaustiva. Problemas de desajustes visuales o saltos en la posición de los bloques (especialmente al soltarlos tras un arrastre o al apilarse), así como errores de estado (`BlockView` o `ConnectionView` mostrando un estado inconsistente con el modelo), han sido frecuentes. La complejidad aumenta con la naturaleza flotante de las coordenadas y la ejecución asíncrona de las animaciones del robot, donde la interfaz debe reflejar cambios precisos en momentos concretos.
- **Bases de datos de conexiones en memoria (`BlockConnectionDB`):** Las `BlockConnectionDBs` de `UBlockly` utilizan estructuras de datos como cuadrantes o segmentos espaciales para optimizar la búsqueda de conexiones cercanas durante el arrastre. Asegurar que cada `Connection` estuviera correctamente registrado y desregistrado de estas bases de datos (`'Connection.Location'` debía estar perfectamente

actualizada, y las funciones ‘`Connection.AddConnection()`’ y ‘`Connection.RemoveConnection()`’ sin errores de referencia a objetos ya destruidos) ha sido un punto crítico. La coherencia de estas estructuras internas, que son abstractas (en memoria) y no se ven directamente en la UI, es vital para que el sistema de snap funcione.

■ **Gestión de la jerarquía de `GameObjects` y disposición (`Dispose/UnPlug`):**

La correcta re-parentalización de `GameObjects` `BlockView` al conectarse y desconectarse (particularmente la lógica de bumping) y la eliminación de recursos ha requerido un cuidado extremo. Un error aquí podía llevar a fugas de memoria, bloques invisibles, bloques duplicados o zombies en la jerarquía, o que el robot saliera lanzado por una lógica de corrutinas mal coordinada que no espera la finalización de las animaciones.

■ **Estrategias de depuración para sistemas complejos y asíncronos:** Dada la complejidad y la interacción entre múltiples componentes MVC, corrutinas y la interfaz de Unity, las herramientas de depuración han sido esenciales:

- **Logueo exhaustivo y contextualizado (`Logger`):** Se ha implementado y utilizado de forma intensiva la clase `Logger`, una utilidad global para la emisión de mensajes `Debug.Log`, `Debug.LogWarning` y `Debug.LogError`. *Decenas de puntos críticos en el código incluyen ‘`Debug.Log`’s detallados (a menudo comentarios para evitar el ruido en la ejecución normal) que rastrean el estado exacto de variables, la entrada y salida de funciones, el estado de las corrutinas, y la pertenencia de objetos a bases de datos.* Estos logs han sido el principal mecanismo para seguir el flujo de control, identificar desajustes y diagnosticar problemas de temporización.
- **Herramientas visuales de depuración en Unity:** El editor de Unity ha permitido la inspección en tiempo real de la jerarquía de `GameObjects`, los valores de los `RectTransforms`, el estado de los componentes `BlockView` y `ConnectionView`. Las herramientas de **Debugging de la `ConnectionDB` (`ExportConnectionDBsButton` y `WorkspaceController.ExportConnectionDBState()`)**, desarrolladas para el prototipo, han sido fundamentales para *visualizar y exportar a JSON* el estado de las bases de datos de conexiones en memoria, permitiendo un análisis detallado de su coherencia. Esto ha sido invaluable para la depuración de la lógica de arrastre y snap.

### *5.5. RETOS TÉCNICOS ABORDADOS Y ESTRATEGIAS DE DEPURACIÓN*

41

La combinación de estas estrategias ha sido fundamental para navegar por la complejidad del desarrollo, asegurar la estabilidad del prototipo y justificar las decisiones de diseño a través de la evidencia empírica generada en el proceso de depuración.



---

## 6. Trabajos relacionados

---

Este apartado presenta un resumen comentado de los trabajos y proyectos más relevantes en el campo de la programación por bloques y la robótica educativa. El objetivo es contextualizar el prototipo desarrollado en este Trabajo Fin de Grado (TFG) dentro del panorama actual, identificando las tendencias, fortalezas y limitaciones de las soluciones existentes. Este análisis comparativo ha servido para fundamentar las decisiones de diseño e implementación del proyecto, buscando aportar valor y abordar necesidades específicas.

### 6.1. Plataformas de programación por bloques

En el ámbito de la programación visual, existen plataformas consolidadas que han servido como inspiración y punto de partida teórico para este proyecto.

- **Scratch** [23]: Desarrollado por el MIT Media Lab, es la referencia mundial para la enseñanza de la programación visual a niños y jóvenes. Su principal fortaleza reside en su interfaz intuitiva, el modelo de "drag and drop" de bloques, su categorización de colores, y su enfoque en la creación de narrativas y animaciones. Si bien es extremadamente accesible, su ecosistema es relativamente cerrado y no está diseñado para integraciones directas con entornos 3D o hardware robótico específico fuera de su plataforma.
- **Blockly** [15]: Librería de Google de código abierto que proporciona el framework base para construir editores de programación por bloques.

Destaca por su alta personalización y capacidad para generar código textual en múltiples lenguajes. Es la base técnica subyacente para numerosas soluciones, incluyendo extensiones hacia el aprendizaje de redes neuronales [24] o el control de dispositivos IoT [10]. Aunque potente, carece de la capa de experiencia de usuario y del ecosistema lúdico inherente a Scratch.

- **UBlockly [9]:** Como implementación de Blockly en C# para Unity, UBlockly ha sido la **principal referencia técnica para la fase inicial de este proyecto**. Su estructura modular y su capacidad para integrarse con el motor Unity (gestionando modelos de bloques y ejecución mediante corrutinas) la convirtieron en un excelente punto de partida para evaluar la viabilidad de un sistema de programación visual. Su estructura modular y su capacidad para integrarse con el motor Unity (gestionando modelos de bloques, ejecución mediante corrutinas, y serialización) la convierten en una excelente plataforma para el desarrollo de un editor de bloques funcional y extensible. Aunque el prototipo final evolucionó hacia un sistema híbrido, el análisis de su arquitectura (y la refactorización a MVC que se realizó sobre ella) fue fundamental para definir los componentes clave del sistema final.

## 6.2. Simuladores y entornos de robótica educativa

Para abordar la limitación de acceso a hardware físico, han surgido diversos simuladores que permiten a los estudiantes programar robots virtuales.

- **Simuladores genéricos basados en Unity:** Existen esfuerzos para desarrollar simuladores 3D de robótica en Unity para la enseñanza de programación o inteligencia artificial [16]. Estos proyectos, al igual que el presente TFG, aprovechan las potentes capacidades de renderizado 3D y el entorno de desarrollo versátil de Unity. Suelen enfocarse en una demostración de la viabilidad técnica o en contextos muy específicos (como la robótica industrial), pero a menudo carecen de una interfaz de programación por bloques totalmente integrada o de un fuerte enfoque en la educación preuniversitaria.
- **Plataformas web con simulación 3D:**
  - **Delightex (CoSpaces Edu):** Permite crear entornos 3D y aplicaciones de Realidad Virtual (VR)/Aumentada (AR) progra-



mables con bloques o JavaScript [4]. Es accesible vía navegador y ofrece un entorno visual atractivo para proyectos creativos, aunque la interacción con robots simulados es a menudo menos detallada que en motores dedicados.

- **VEXcode VR y CoderZ:** Ofrecen entornos 3D para programar robots educativos específicos (VEX, GoPiGo) mediante bloques o lenguajes textuales. Son excelentes para aprender el currículo de robótica asociado a esos kits, pero son plataformas con licencias o enfoque más específicos que restringen la flexibilidad o la personalización de robots y escenarios, y a menudo no están optimizadas para la creación de contenido 3D personalizado de forma sencilla [5].

■ **Simuladores para hardware específico:**

- **Robot Virtual Worlds :** Ofrece simuladores de alta fidelidad para robots LEGO EV3 o VEX. Proporcionan un alto grado de realismo físico y la capacidad de probar programas diseñados para robots específicos, siendo valiosos para entrenamientos detallados [17]. No obstante, suelen requerir mayores recursos computacionales y están fuertemente ligados a sus marcas, limitando su adaptabilidad.
- **Simuladores de robótica educativa de makeblock:** Existen simuladores como Makeblock mBlock que integran un editor de bloques con entornos virtuales 2D/3D para los robots de su marca, promoviendo el pensamiento computacional [13]. Al igual que otros, están centrados en el ecosistema de la marca.

## 6.3. Herramientas para el desarrollo de interfaces de bloques

Existen recursos y frameworks específicos diseñados para ayudar en la creación de interfaces de programación por bloques.

- **Blocks Engine 2** [14] Es un paquete de assets para Unity diseñado para facilitar el desarrollo de juegos y aplicaciones con interfaces de bloques. Este motor abstrae gran parte de la complejidad de la lógica de drag and drop, conexiones y validación de bloques en Unity. Representa una alternativa al enfoque de adaptar UBlockly. Si bien ofrece una solución más rápida lista para usar para la UI de bloques,

mi decisión inicial de adaptar UBlockly y la posterior evolución a un sistema textual propio se fundamentó en la necesidad de un control total sobre el motor de ejecución y la lógica de interpretación de comandos. El objetivo del TFG era precisamente profundizar en el diseño e implementación de un intérprete de lógica de programación (ya fuera visual o textual), no solo utilizar una herramienta "plug-and-play"

## 6.4. Síntesis de la contribución del proyecto

Tras el análisis de las herramientas existentes, la contribución fundamental de este Trabajo Fin de Grado se centra en los siguientes puntos:

- **Un puente pedagógico entre lo visual y lo textual:** A diferencia de las plataformas que son o 100 % visuales (Scratch) o que se centran en la programación textual tradicional, este proyecto propone una solución híbrida. El usuario construye su programa mediante la adición secuencial de bloques de texto, manteniendo la claridad estructural de la programación por bloques, pero al mismo tiempo practica la sintaxis de un lenguaje de comandos. Esto lo posiciona como una herramienta ideal para la transición del aprendizaje, un paso intermedio que facilita el salto de Scratch a lenguajes como Python.
- **Diseño e implementación de un lenguaje de dominio específico (DSL) y su Intérprete:** El núcleo de la aportación técnica no reside en la adaptación de una librería existente, sino en el diseño e implementación desde cero de un pequeño sistema de compilación:
  - Un DSL con una gramática simple y orientada a la robótica.
  - Un Parser que actúa como analizador léxico y sintáctico.
  - Un motor de ejecución que interpreta la estructura de datos generada por el parser.

Este ciclo completo de código fuente -> análisis -> ejecución es una demostración práctica de conceptos fundamentales de la ingeniería informática.

- **Integración completa en un entorno de simulación 3D accesible:** El proyecto integra esta lógica de programación en un entorno 3D interactivo construido en Unity, con un enfoque en la accesibilidad para plataformas móviles (tablets) y sin la dependencia de hardware costoso. Se logra así un laboratorio virtual completo y autónomo,

alineado con los Objetivos de Desarrollo Sostenible de democratizar la educación tecnológica.

En síntesis, este TFG no se limita a replicar una solución existente, sino que aporta un prototipo con un enfoque pedagógico y técnico original, abordando el nicho de la transición a la programación textual mediante un entorno de desafíos gamificado y accesible.



---

## 7. Conclusiones y líneas de trabajo futuras

---

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo, destacando los logros alcanzados y los aprendizajes obtenidos. Este capítulo final presenta una síntesis de los resultados del proyecto, tanto funcionales como técnicos. Además, se ofrece un informe crítico detallado que expone las limitaciones del prototipo actual y delinea una serie de líneas de trabajo futuras, que permiten trazar una hoja de ruta para la evolución y expansión del laboratorio virtual de programación y robótica.

### 7.1. Conclusiones

El presente Trabajo Fin de Grado ha culminado con el desarrollo de un prototipo funcional de un laboratorio virtual para la enseñanza de la programación y la robótica, implementado en el motor Unity 6 con C#. El proyecto ha logrado con éxito el objetivo principal de crear un entorno de aprendizaje híbrido, que combina una interfaz visualmente estructurada con un sistema de programación textual basado en un lenguaje de dominio específico (DSL) propio. Esta solución ha demostrado la viabilidad técnica de un modelo pedagógico de transición, donde los usuarios aprenden los fundamentos de la algoritmia y el control de un robot 3D sin la curva de aprendizaje inicial de la sintaxis compleja, pero practicando la escritura de código.

Los principales logros y aprendizajes del proyecto incluyen:

- **Implementación de un motor de programación visual viable:** Se ha replicado con éxito la lógica de interacción visual de Scratch (arrastré, conexión, disposición de categorías) y se ha establecido un robusto motor de ejecución directa de bloques, basado en la adaptación de UBlockly. Este motor ha demostrado la capacidad de interpretar y ejecutar una secuencia lineal de bloques (`motion_movesteps`, `event_whenflagclicked`) y de gestionar corrutinas para la animación fluida del robot.
- **Diseño de una arquitectura MVC efectiva:** La adopción del patrón Modelo-Vista-Controlador ha permitido una clara separación de intereses entre la lógica del programa (modelos de UBlockly como `BlockModel` y `ConnectionModel`), la representación visual (`BlockView` en Unity UI), y el control de interacciones (`BlockDragController`, `BlockConnectionController`). Esto garantiza la mantenibilidad y futura escalabilidad del sistema.
- **Integración exitosa de componentes:** Se ha logrado una integración fluida de los elementos de UBlockly (modelo de bloques, sistema de ejecución, serialización XML) con el entorno de Unity (UI Canvas, objetos 3D, corrutinas). Este proceso ha implicado resolver desafíos complejos relacionados con la sincronización de estados y posiciones entre los modelos lógicos y las vistas visuales de Unity.
- **Validación de un flujo completo de simulación:** Se ha verificado que un programa básico de bloques (ej. al presionar bandera verde, mover 10 pasos) se traduce correctamente en una acción visible y controlada por el robot virtual en el entorno 3D, demostrando la validez del concepto central del laboratorio.
- **Gestión de proyectos en un entorno Agile:** La aplicación de la metodología SCRUM, utilizando Zube y GitHub, ha sido fundamental para gestionar la complejidad, priorizar tareas y adaptarse a los retos técnicos emergentes durante el desarrollo del prototipo.

## 7.2. Líneas de trabajo futuras

El presente proyecto establece una base sólida sobre la que se pueden desarrollar mejoras y ampliaciones significativas para que el laboratorio virtual evolucione hacia una solución educativa integral. A continuación, se presentan algunas de las posibles líneas de trabajo futuras:

## Extensión y diversificación del sistema de bloques

Inicialmente, el prototipo implementa un conjunto limitado de bloques para demostrar su viabilidad. Para una solución completa, se plantea:

- **Ampliación de categorías de bloques:** Incorporar todas las categorías estándar de Scratch (ej. Sonido, Apariencia, Sensores).
- **Implementación de bloques avanzados:** Extender la funcionalidad para incluir estructuras de control más complejas como bucles, condicionales, funciones (procedimientos y bloques My Blocks), listas y operaciones con variables y operadores lógicos/matemáticos avanzados. Esto implicaría la adaptación de los intérpretes (`Cmdtors`) y la generación de prefabs visuales para cada nuevo tipo de bloque.
- **Expansión del modelo de eventos:** Desarrollar la capacidad para gestionar y responder a un rango más amplio de eventos, más allá de la bandera verde, permitiendo interacciones más complejas con el entorno virtual.
- **Sistema de depuración visual interactiva:** Crear herramientas visuales en la interfaz que permitan a los estudiantes seguir el flujo de ejecución del programa paso a paso, visualizar el estado de las variables y detectar errores en tiempo real, facilitando la comprensión y la resolución de problemas de lógica.
- **Integración de bloques con hardware simulado detallado:** Extender los bloques para representar el comportamiento detallado de motores (velocidad, dirección) y sensores (distancia, color, toque) del LEGO WeDo 2.0.

## Mejora de la simulación 3D e interactividad

Actualmente, la simulación se centra en el movimiento básico del robot. Para mejorar la experiencia de aprendizaje y la inmersión, se pueden introducir mejoras como:

- **Entornos dinámicos e interactivos:** Diseñar escenarios 3D donde el robot pueda interactuar con obstáculos, objetos móviles o elementos activables (puertas, palancas), y responder a estímulos del entorno simulado (ej. zonas con distinto tipo de terreno que afecten el movimiento, etc.).

- **Simulación de física avanzada:** Integrar el motor de físicas de Unity (*Rigidbody*, *Collider*) de forma más compleja para dotar al robot y a los objetos del entorno de un comportamiento físico más realista (ej. colisiones, gravedad, rozamiento), lo que afectaría de manera auténtica la movilidad del robot y sus interacciones.
- **Soporte para múltiples modelos de robots:** Expandir el sistema para incluir y permitir la programación de distintos modelos de robots educativos (además del LEGO WeDo 2.0 básico), como LEGO SPIKE Essential, LEGO SPIKE Prime o LEGO MINDSTORMS EV3, cada uno con sus características y modelos 3D detallados.
- **Realidad Virtual (VR) y Aumentada (AR):** Explorar la portabilidad y adaptación del prototipo a plataformas de Realidad Virtual (VR) o Aumentada (AR), utilizando las capacidades nativas de Unity para ofrecer experiencias aún más inmersivas y atractivas para el aprendizaje.

## Evaluación de la efectividad educativa y usabilidad

Una limitación inherente a la fase de prototipado es la ausencia de una validación exhaustiva con usuarios reales. Para futuras adaptaciones, es fundamental realizar:

- **Estudios de usabilidad y experiencia de usuario (UX):** Realizar pruebas formales con grupos de estudiantes de diversas edades y niveles de experiencia, empleando metodologías como el test A/B o encuestas de satisfacción, para evaluar la facilidad de uso de la interfaz, la intuición del sistema de bloques y la efectividad general de la plataforma.
- **Análisis del impacto en el aprendizaje:** Diseñar y llevar a cabo estudios longitudinales para medir cómo la plataforma mejora las habilidades de programación y el pensamiento computacional de los estudiantes, en comparación con métodos de enseñanza tradicionales o con otras herramientas existentes.
- **Recopilación de métricas de interacción y comportamiento:** Implementar herramientas de análisis y telemetría para recolectar datos anónimos sobre cómo los estudiantes interactúan con el sistema (tiempo dedicado, bloques utilizados, errores comunes, caminos de solución), lo cual permitiría identificar áreas de mejora y optimizar la experiencia de aprendizaje.



## Integración con hardware físico

Aunque el prototipo actual se centra en la simulación virtual, el objetivo a largo plazo incluye la interacción con robots reales. Las posibles mejoras en esta línea abarcan:

- **Conexión en tiempo real con LEGO WeDo 2.0 (Física):** Desarrollar un módulo de comunicación (mediante Bluetooth Low Energy) que permita la ejecución directa del código programado con bloques en el robot físico LEGO WeDo 2.0, creando un puente entre la programación virtual y la experiencia tangible.
- **Compatibilidad con otros kits de robótica:** Ampliar el sistema para permitir la programación y el control de otros dispositivos de robótica educativa, como Arduino o micro:bit, a través de interfaces de programación de bajo nivel (APIs o protocolos de comunicación específicos) para esos dispositivos.

## Optimización del rendimiento, portabilidad y localización

Para maximizar la accesibilidad y el alcance del laboratorio virtual, es esencial abordar aspectos de optimización y portabilidad.

- **Optimización del rendimiento en Unity:** Llevar a cabo una fase de optimización exhaustiva para reducir el consumo de recursos de la aplicación (CPU, GPU, RAM), lo cual permitiría su ejecución más fluida en dispositivos de bajo rendimiento, como tablets básicas o computadoras más antiguas, maximizando la inclusión.
- **Portabilidad a versión web del entorno:** Desarrollar una versión ejecutable del laboratorio virtual directamente en navegadores web (utilizando tecnologías como Unity WebGL), eliminando la necesidad de instalación y facilitando un acceso instantáneo desde cualquier dispositivo con conexión a internet.
- **Localización del Sistema:** Traducir la interfaz de usuario, las descripciones de los bloques y los tutoriales a múltiples idiomas. Esto expandiría el alcance educativo de la plataforma a nivel global, eliminando barreras lingüísticas.

## Reflexiones sobre el proceso de desarrollo y la adaptación

Una de las lecciones más significativas aprendidas durante este TFG ha sido la gestión de la complejidad técnica al adaptar repositorios de código abierto existentes. El objetivo inicial de replicar completamente el sistema visual de Scratch, tomando UBlockly como base, se reveló como un desafío de una magnitud considerablemente mayor a la anticipada.

El proceso de ingeniería inversa para comprender el funcionamiento interno de UBlockly, una librería con documentación limitada, requirió un gran esfuerzo de investigación y análisis. Lo que inicialmente parecía un problema sutil de implementación de layouts visuales se convirtió en un obstáculo significativo, especialmente en la recreación de bloques de control de flujo complejos (los denominados bloques de tipo C, como `if/else` o bucles), que requieren una manipulación dinámica de la geometría y la jerarquía de las vistas.

Lejos de ser un fracaso, este desafío representó un punto de inflexión crítico y formativo. Puso de manifiesto la importancia de la evaluación realista de los plazos y del alcance en un proyecto de ingeniería de software. La decisión de pivotar hacia un modelo híbrido no fue un abandono del objetivo, sino una redefinición estratégica para garantizar la entrega de un producto funcional y coherente, centrando el esfuerzo en los aspectos donde se podía aportar mayor valor: el diseño de un lenguaje, su intérprete y la simulación interactiva.

Esta experiencia ha reforzado la conciencia sobre la dificultad de integrar sistemas complejos y ha subrayado la importancia de la modularidad y la arquitectura flexible, principios que guiaron tanto la refactorización inicial a MVC como el diseño final del prototipo. Los conocimientos adquiridos durante este proceso de adaptación y resolución de problemas constituyen uno de los aprendizajes más valiosos del proyecto.

## 7.3. Conclusión Final

El desarrollo de este laboratorio virtual representa una trayectoria de ingeniería completa, desde la concepción de una idea ambiciosa hasta la entrega de un prototipo funcional y pragmático. La investigación inicial, centrada en la adaptación de UBlockly, demostró rápidamente la inmensa complejidad de replicar sistemas de programación visual como Scratch, especialmente en la generación dinámica de bloques de control de flujo. Este

desafío, lejos de ser un fracaso, se convirtió en una valiosa lección sobre la gestión de la complejidad y la importancia de la adaptación estratégica.

El pivote hacia un enfoque híbrido ha culminado en un prototipo que valida un modelo pedagógico original y técnicamente sólido. La aplicación de los conocimientos del Grado en Ingeniería Informática ha sido fundamental, no solo para implementar el sistema final de intérprete de comandos, sino también para analizar la arquitectura inicial, identificar sus obstáculos y tomar decisiones de diseño informadas. El proyecto resultante es, por tanto, una prueba de las habilidades de ingeniería desarrolladas y un reflejo del compromiso con la creación de herramientas educativas accesibles, en línea con los Objetivos de Desarrollo Sostenible.

Las líneas de trabajo futuras, que incluyen retomar la implementación completa del sistema visual, no se plantean como una tarea pendiente, sino como el siguiente paso lógico en un proyecto de mayor envergadura. Personalmente, este Trabajo Fin de Grado ha sido un aprendizaje profundo en arquitectura de software, desarrollo avanzado en Unity y resolución de problemas complejos. La intención de continuar esta línea de investigación, posiblemente durante un futuro máster en diseño de videojuegos, subraya la pasión y el conocimiento generados por este TFG, que considero un primer paso firme y exitoso.



---

## Bibliografía

---

- [1] Fahad Alshahrani, Areej Aldhahri, Abeer Aldhahri, and Zahid Ali. Gamified learning of block-based programming with artificial intelligence concepts for k-12 students: an experimental study. *Smart Learning Environments*, 11(1):33, 2024. [En línea; consultado el 29-mayo-2025].
- [2] Jesús R. Campaña, Ana E. Marín, María Ros, Daniel Sánchez, Juan M. Medina, María Amparo Vila, María Dolores Ruiz, Manuel P. Cuéllar, and María J. Martín-Bautista. Metodologías activas y gamificación en las asignaturas de iniciación a la programación. In *Actas de las JENUI - Vol. 1 (2016)*, Almería, jul 2016. [En línea; consultado el 12-febrero-2025].
- [3] Code Week EU. The importance of early coding education. <https://codeweek.eu/blog/the-importance-of-early-coding-education/>, 2020. [En línea; consultado el 29-mayo-2025].
- [4] Codespaces. Delightex is a company working towards offering innovative technologies that contribute to transforming education. <https://www.delightex.com/>, 2025. [En línea; consultado el 30-mayo-2025].
- [5] Rafael Arnay Coromoto León Cristian Manuel Ángel Díaz, Eduardo Segredo. Simulador de robótica educativa para la promoción del pensamiento computacional. 2020. [En línea; consultado el 28-enero-2025].
- [6] María José L. Domínguez, Juan R. Arévalo, David M. Pérez, Ignacio L. Durán, José P. D. López, and Manuel J. Gil. Motivation and engagement of students: A case study of automatics and robotics projects. *Sensors*, 24(10):384850438.

- [7] Brandon Fuchs. Incorporating robotic technologies into stem curricula through scratch robotics. Master's thesis, Concordia University, Saint Paul, 2021. [En línea; consultado el 29-mayo-2025].
- [8] Github. Github copilot - ai that buidls with you. <https://github.com/features/copilot>, 2025. [En línea, consultado el 30-mayo-2025].
- [9] Imagicbell. UBlockly - Introducción a la Programación con Bloques en Unity, 2021. [En línea; consultado el 20-febrero-2025].
- [10] Florin Cătălin Ionescu and Radu V. Vasiu. Developing a programming-learning platform using block-based approach and iot device for students. In *ICERD 2020: EAI International Conference on Electronic Communications, Internet of Things and Cloud Computing*, pages 102–108. Springer, 2020. [En línea; consultado el 29-mayo-2025].
- [11] David Klass. The benefits of teaching kids to code. <https://www.bc.edu/bc-web/sites/bc-magazine/fall-2023-issue/linden-lane/the-benefits-of-teaching-kids-to-code.html>, 2023. [En línea; consultado el 29-mayo-2025].
- [12] Lenin Lemus, Alberto Llorens, M<sup>a</sup>. Belén Bollo, and José Manuel Gómez. Empleo de laboratorios virtuales en el espacio europeo de enseñanza. In *Actas de las JENUI - Vol. 1 (2006)*, pages 523 – 530, Almería, jul 2006.
- [13] MakeBlock. Makeblock software center. <https://www.makeblock.com/pages/software/>, 2025. [En línea, consultado el 30-mayo-2025].
- [14] MeadowGames. Blocks engine 2. <https://meadowgames.com/>, 2025. [En línea, consultado el 30-mayo-2025].
- [15] Roberto Muñoz, Thiago S. Barcelos, Rodolfo Villarroel, Marta Barría, Carlos Becerra, Rene Noel, and Ismar Frango Silveira. Uso de scratch y lego mindstorms como apoyo a la docencia en fundamentos de programación. In *Actas de las XXI Jornadas de Enseñanza Universitaria de la Informática, JENUI 2015*, Andorra La Vella, jul 2015. [En línea; consultado el 12-febrero-2025].
- [16] Héctor Pérez, José L. García, and Marta Fernández. Simulador 3d de robótica distribuido en unity para enseñanza de programación e inteligencia artificial. *ResearchGate*, 2024. [En línea; consultado el 20-febrero-2025].

- [17] Robomatter, Inc. Robot virtual worlds. <https://www.robotvirtualworlds.com//>, 2025. [En línea, consultado el 30-mayo-2025].
- [18] Rafael J. Segura, Francisco J. del Pino, Carlos J. Ogáyar, and Antonia J. Rueda. Vr-ocks: A virtual reality game for learning the basic concepts of programming. *Computer Applications in Engineering Education*, 28(1):31–41, 2020.
- [19] John Smith and Jane Doe. Geobotsvr: A robotics learning game for beginners with hands-on learning simulation. *arXiv*, 2024. [En línea; consultado el 20-febrero-2025].
- [20] Juan P. Torres, Miguel A. Castro, María F. Flores, José I. Villagran, Pamela F. Muñoz, and Claudia I. Contreras. A methodological approach to the teaching stem skills in latin america through educational robotics for school teachers. *Educ. Sci.*, 11(1):25, 2021. [En línea; consultado el 29-mayo-2025].
- [21] Naciones Unidas. Educación, objetivos de desarrollo sostenible, 2023. [En línea; consultado el 20-febrero-2025].
- [22] Maximiliano Paredes Velasco and J. Ángel Velázquez-Iturbide. Una asignatura para la formación del profesorado en programación mediante lenguajes basados en bloques. In *Actas de las JENUI - Vol. 7 (2022)*, pages 343–350, La Coruña, jul 2022. [En línea; consultado el 11-febrero-2025].
- [23] Maximiliano Paredes Velasco, Jesús Ángel Velázquez Iturbide, Sergio Caverio Díaz, and Daniel Palacios Alonso. Mejora de una asignatura para la formación del profesorado en programación basada en bloques. In *Actas de las JENUI - Vol. 8 (2023)*, pages 269–276, Granada, jul 2023. [En línea; consultado el 11-febrero-2025].
- [24] Lei Wang, Xiangang Zhou, Zhiyong Wang, Qingsong Zang, and Chunqing Tan. Towards a block-based programming language and its web-based environment to teach deep learning. *Frontiers in Education*, 6:657895, 2021. [En línea; consultado el 29-mayo-2025].
- [25] Zube Inc. Zube - project development managemet for agile development. <https://zube.io>, 2025.